

1939. POP3 es un protocolo más simple, pero soporta menos características y es menos seguro en el uso común. Por lo general el correo se descarga a la computadora del agente de usuario, en vez de permanecer en el servidor de correo. Esto facilita la vida al servidor, pero es más difícil para el usuario. No es fácil leer el correo en varias computadoras; además, si la computadora del agente de usuario se descompone, cabe la probabilidad de perder todo el correo electrónico de manera permanente. Sin embargo, el protocolo POP3 se sigue utilizando en muchas partes.

También se pueden usar protocolos propietarios, debido a que el protocolo se ejecuta entre un servidor de correo y un agente de usuario que pueden ser suministrados por la misma empresa. Microsoft Exchange es un sistema de correo con un protocolo propietario.

Correo web

Una alternativa cada vez más popular a IMAP y SMTP para proveer servicio de correo electrónico es utilizar la web como interfaz para enviar y recibir correo. Los sistemas de *correo web* que se utilizan ampliamente son Google Gmail, Microsoft Hotmail y Yahoo! Mail. El correo web es un ejemplo de software (en este caso, un agente de usuario de correo) que se proporciona como servicio mediante el uso de la web.

En esta arquitectura, el proveedor ejecuta los servidores de correo de la manera usual para aceptar los mensajes para los usuarios, con SMTP en el puerto 25. Sin embargo, el agente de usuario es distinto. En vez de ser un programa independiente, es una interfaz de usuario que se provee a través de páginas web. Esto significa que los usuarios pueden usar el navegador que deseen para acceder a su correo y enviar nuevos mensajes.

Aún no hemos estudiado la web, pero a continuación le daremos una breve descripción que puede utilizar como referencia en lo futuro. Cuando el usuario visita la página web de correo electrónico del proveedor, aparece un formulario en el que se le pide que introduzca un nombre de inicio de sesión y una contraseña. El nombre de inicio de sesión y la contraseña se envían al servidor, quien procede a validarlas. Si el inicio de sesión es exitoso, el servidor encuentra el buzón de correo del usuario y crea una página web en la que lista el contenido del buzón de correo al instante. Después, la página web se envía al explorador para que la muestre en pantalla.

Es posible hacer clic en muchos de los elementos en la página que muestra el buzón de correo, de modo que los mensajes se puedan leer, eliminar, etc. Para que la interfaz sea receptiva, es común que las páginas web incluyan programas de JavaScript. Estos programas se ejecutan en forma local en el cliente, en respuesta a los eventos locales (por ejemplo, los clics del ratón); además pueden descargar y enviar mensajes en segundo plano, para preparar el siguiente mensaje a visualizar o un nuevo mensaje para su envío. En este modelo, el envío de correo ocurre mediante el uso de los protocolos web normales, en donde se publican datos a un URL. El servidor web se hace cargo de inyectar los mensajes al sistema tradicional de entrega de correo que hemos descrito. Por razones de seguridad, también se pueden usar los protocolos web estándar. Estos protocolos se encargan del cifrado de las páginas web y no les importa si el contenido de éstas es un mensaje de correo.

7.3 WORLD WIDE WEB

La web, popularmente conocida como World Wide Web, es un marco arquitectónico para acceder a cierto contenido vinculado distribuido en millones de máquinas por toda Internet. En 10 años pasó de ser una manera de coordinar el diseño de los experimentos de física de alta energía en Suiza a la aplicación que millones de personas piensan que es “Internet”. Su enorme popularidad se deriva del hecho de que es fácil

de usar para los principiantes, además de que provee acceso mediante una interfaz gráfica a un enorme cúmulo de información sobre casi cualquier tema concebible, desde aborígenes hasta zoología.

La web (también conocida como WWW) comenzó en 1989 en el CERN, el Centro Europeo de Investigación Nuclear. La idea inicial era ayudar a los equipos grandes, a menudo con miembros en media docena de países y zonas horarias o más, a que colaboraran mediante el uso de un conjunto siempre cambiante de informes, planos, dibujos, fotografías y demás documentos producidos por los experimentos de partículas en física. La propuesta de una red de documentos vinculados surgió de Tim Berners-Lee, físico del CERN. El primer prototipo (basado en texto) estaba en operación 18 meses después. Una demostración pública en la conferencia Hypertext '91 atrajo la atención de otros investigadores, lo que llevó a Marc Andreessen, de la Universidad de Illinois, a desarrollar el primer navegador gráfico. Se llamó Mosaic y se liberó en febrero de 1993.

El resto, como dicen, es historia. Mosaic fue tan popular que un año más tarde, Andreessen formó su propia compañía llamada Netscape Communications Corp., cuya meta era desarrollar software web. Durante los siguientes tres años, Netscape Navigator y Microsoft Internet Explorer sostuvieron una “guerra de navegadores”, en donde cada uno trataba de capturar una mayor parte del nuevo mercado al agregar frenéticamente más características (y, por ende, más errores) que el otro.

Durante las décadas de 1990 y 2000, los sitios web y las páginas web, como se le dice al contenido web, crecieron en forma exponencial hasta que hubo millones de sitios y miles de millones de páginas. Una pequeña cantidad de estos sitios se hicieron muy populares. Esos sitios y las empresas que los respaldan son los que definen en gran parte la web y la forma en que las personas la experimentan en la actualidad. Algunos ejemplos son: una librería (Amazon, empezó en 1994, capitalización de mercado de \$50 mil millones), un sitio de subastas en línea (eBay, 1995, \$30 mil millones), un sitio de búsqueda (Google, 1998, \$150 mil millones) y redes sociales (Facebook, 2004, empresa privada valuada en más de \$15 mil millones). Incluso el periodo que atraviesa el año 2000, cuando el valor de muchas empresas web aumentó hasta cientos o millones de dólares de la noche a la mañana, sólo para derrumbarse por completo al siguiente día cuando resultaron ser un mero despliegue publicitario, tiene su nombre. Se conoce como la **Era punto com**. Aún siguen surgiendo nuevas ideas que se convierten en un negocio multimillonario en la web. Muchas de ellas provienen de estudiantes. Por ejemplo, Mark Zuckerberg era un estudiante de Harvard cuando empezó Facebook, Sergey Brin y Larry Page eran estudiantes en Stanford cuando empezaron Google. Tal vez usted descubrirá la siguiente idea multimillonaria.

En 1994, el CERN y el MIT firmaron un acuerdo para establecer el **W3C (Consorcio World Wide Web)**, del inglés *World Wide Web Consortium*, una organización dedicada al desarrollo de web, a la estandarización de protocolos y a fomentar la interoperabilidad entre los sitios. Berners-Lee se convirtió en el director. Desde entonces, cientos de universidades y compañías se han unido al consorcio. Aunque ahora hay más libros sobre la web de los que podemos contar, el mejor lugar para recibir información actualizada sobre ella es (naturalmente) la web misma. La página de inicio del consorcio se encuentra en www.w3.org. Los lectores interesados pueden encontrar ahí vínculos con páginas que cubren todos los numerosos documentos y actividades del consorcio.

7.3.1 Panorama de la arquitectura

Desde el punto de vista del usuario, la web consiste en una enorme colección de contenido en forma de **páginas web**, por lo general, conocidas simplemente como **páginas**. Cada una puede contener vínculos a otras páginas en cualquier lugar del mundo. Para seguir un vínculo, los usuarios pueden hacer clic en él, y a continuación los llevará a la página apuntada. Este proceso se puede repetir de manera indefinida. La idea de hacer que una página apunte a otra, lo que ahora se conoce como **hipertexto**, fue inventada por un profesor visionario de ingeniería eléctrica del MIT, Vannevar Bush, en 1945 (Bush, 1945). Esto fue

mucho antes de que se inventara Internet. De hecho, fue antes de que existieran las computadoras comerciales, aunque varias universidades habían producido prototipos que ocupaban habitaciones extensas pero tenían menos poder que una moderna calculadora de bolsillo.

Por lo general, las páginas se ven mediante un programa llamado **navegador**. Firefox, Internet Explorer y Chrome son ejemplos de navegadores populares. El navegador obtiene la página solicitada, interpreta el contenido y despliega la página en pantalla con el formato adecuado. El contenido en sí puede ser una mezcla de texto, imágenes y comandos de formato, ya sea en forma de un documento tradicional u otras formas de contenido, como un video o programas que produzcan una interfaz gráfica con la que puedan interactuar los usuarios.

En la parte superior izquierda de la figura 7-18 se muestra la imagen de una página. Ésta es la página del Departamento de Ciencias Computacionales e Ingeniería, de la Universidad de Washington. Esta página muestra texto y elementos gráficos (que en su mayoría son muy pequeños como para leerlos). Algunas partes de la página están asociadas con vínculos a otras. A una pieza de texto, icono, imagen u otro elemento asociado con otra página se le conoce como **hipervínculo**. Para seguir un vínculo, el usuario coloca el cursor del ratón en la parte vinculada del área de la página (lo cual hace que el cursor cambie de forma) y hace clic. Seguir un vínculo es simplemente una forma de decir al navegador que obtenga otra página. En los primeros días de la web, los vínculos se resaltaban con texto subrayado y coloreado para que pudieran sobresalir. Hoy en día, los creadores de las páginas web tienen formas de controlar la apariencia de las regiones vinculadas, por lo que un vínculo podría aparecer como un icono o cambiar su apariencia cuando el ratón pase sobre él. Queda a criterio de los creadores de la página hacer que los vínculos se puedan distinguir en forma visual para proveer una interfaz que se pueda usar.

Los estudiantes en el Departamento pueden aprender más si siguen un vínculo a una página con información especial para ellos. Para acceder a este vínculo deben hacer clic en el área encerrada en un círculo. A continuación el navegador obtiene la nueva página y la despliega en pantalla, como se muestra de manera parcial en la parte inferior izquierda de la figura 7-18. Hay docenas de páginas adicionales a las que se puede acceder mediante vínculos desde la primera página, además de este ejemplo. Cualquier otra página

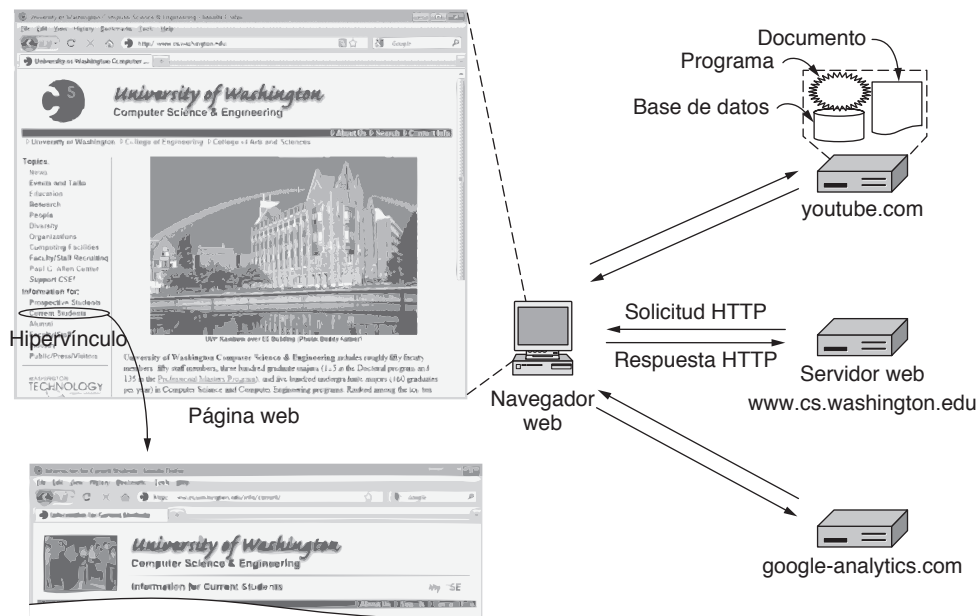


Figura 7-18. Arquitectura de la web.

puede estar compuesta de contenido en la(s) misma(s) máquina(s) que la primera página, o en máquinas en alguna parte del mundo. El usuario no puede distinguir esto. El navegador se encarga del proceso de obtención de las páginas, sin ninguna ayuda del usuario. De esta forma, el proceso de desplazarse entre máquinas al momento de ver contenido es transparente para el usuario.

En la figura 7-18 se muestra el modelo básico de la forma en que se despliegan las páginas en pantalla. El navegador despliega una página web en la máquina cliente. Para obtener cada página, se envía una solicitud a uno o más servidores, los cuales responden con el contenido de la página. El protocolo de solicitud-respuesta para obtener páginas es un protocolo simple basado en texto que se ejecuta sobre TCP, como en el caso de SMTP. Este protocolo se llama **HTTP (Protocolo de Transferencia de Hipertexto)**, del inglés *HyperText Transfer Protocol*). El contenido puede ser simplemente un documento que se lea de un disco, o el resultado de una consulta en una base de datos y la ejecución de un programa. La página se considera una **página estática** si es el mismo documento cada vez que se despliega en pantalla. Por el contrario, si se generó bajo demanda mediante un programa o contiene uno, es una **página dinámica**.

Una página dinámica se puede presentar de manera distinta cada vez que se despliega en pantalla. Por ejemplo, la página principal de una tienda electrónica puede ser distinta para cada visitante. Si el cliente de una librería ha comprado novelas de misterio en sus anteriores visitas, es probable que la próxima vez le aparezcan las nuevas novelas de misterio en la página de inicio; mientras que un cliente enfocado hacia lo culinario podría ser recibido con nuevos libros de cocina. La manera en que el sitio web mantiene el registro de los gustos de cada cliente es algo que veremos en breve. En sí, la respuesta involucra el uso de *cookies*.

En la figura anterior, el navegador se contacta con tres servidores para obtener las dos páginas: *cs.washington.edu*, *youtube.com* y *google-analytics.com*. El contenido de estos distintos servidores se integra para que el navegador lo despliegue. La visualización conlleva un rango de procesamiento que depende del tipo de contenido. Además de generar texto y gráficos, puede implicar la reproducción de un video o la ejecución de una secuencia de comandos (*script*) que presente su propia interfaz de usuario como parte de la página. En este caso, el servidor *cs.washington.edu* suministra la página principal, el servidor *youtube.com* provee un video incrustado y el servidor *google-analytics.com* no suministra nada que el usuario pueda ver, pero rastrea los visitantes del sitio. Más adelante hablaremos con más detalle sobre los rastreadores.

El lado del cliente

Examinemos ahora el lado del navegador web en la figura 7-18 con más detalle. En esencia, un navegador es un programa que puede mostrar una página web y capturar clics del ratón para los elementos de la página visualizada. Al seleccionar un elemento, el navegador sigue el hipervínculo y obtiene la página seleccionada.

Cuando se creó la web por primera vez, resultó obvio a primera vista que para hacer que una página apuntara a otra página web se requerían mecanismos para nombrar y localizar páginas. En particular, había que responder a tres preguntas para desplegar una página:

1. ¿Cómo se llama la página?
2. ¿En dónde está ubicada?
3. ¿Cómo se puede acceder a ella?

Si a cada página se le asignara de alguna forma un nombre único, no habría ninguna ambigüedad a la hora de identificarlas. Sin embargo, esto no resolvería el problema. Considere un paralelismo entre las personas y las páginas. En Estados Unidos casi todos tienen un número de seguro social, el cual es un identificador único, ya que no debe haber dos personas con el mismo número. Pero si usted sólo cuenta

con el número de Seguro Social, no hay manera de averiguar la dirección del propietario ni tampoco puede saber si debe escribir a la persona en inglés, español o chino. La web tiene básicamente los mismos problemas.

La solución que se eligió identifica a las páginas de una manera que resuelve los tres problemas a la vez. A cada página se le asigna un **URL (Localizador Uniforme de Recursos**, del inglés *Uniform Resource Locator*) que sirva de manera efectiva como el nombre mundial de la página. Los URL tienen tres partes: el protocolo (también conocido como **esquema**), el nombre DNS de la máquina en la que se encuentra la página y la ruta que indica de manera única la página específica (un archivo a leer o un programa a ejecutar en la máquina). En el caso general, la ruta tiene un nombre jerárquico que modela la estructura de un directorio de archivos. Sin embargo, la interpretación de la ruta depende del servidor; puede o no reflejar la estructura del directorio real.

Como ejemplo, el URL de la página que se muestra en la figura 7-18 es:

<http://www.cs.washington.edu/index.html>

Este URL consiste de tres partes: el protocolo (*http*), el nombre DNS del host (*www.cs.washington.edu*) y el nombre de la ruta (*index.html*).

Cuando un usuario hace clic en un hipervínculo, el navegador lleva a cabo una serie de pasos para obtener la página a la que apunta. Vamos a rastrear los pasos que se realizan al momento de seleccionar nuestro vínculo de ejemplo:

1. El navegador determina el URL (al ver lo que se seleccionó).
2. El navegador pide al DNS la dirección IP del servidor *www.cs.washington.edu*.
3. El DNS responde con 128.208.3.88.
4. El navegador realiza una conexión TCP a 128.208.3.88 en el puerto 80, el puerto conocido para el protocolo HTTP.
5. Después envía una solicitud HTTP para pedir la página */index.html*.
6. El servidor *www.cs.washington.edu* envía la página como una respuesta HTTP, por ejemplo, enviando el archivo */index.html*.
7. Si la página incluye los localizadores URL necesarios para desplegar en pantalla, el navegador obtiene los otros URL mediante el mismo proceso. En este caso, los URL incluyen varias imágenes incrustadas que también se obtienen de *www.cs.washington.edu*, así como un video de *youtube.com* y una secuencia de comandos (*script*) de *google-analytics.com*.
8. El navegador despliega la página */index.html* como aparece en la figura 7-18.
9. Se liberan las conexiones TCP si no hay más solicitudes para los mismos servidores durante un periodo corto.

Muchos navegadores despliegan en una línea de estado, que se encuentra en la parte inferior de la pantalla, qué paso están ejecutando en ese momento. De esta manera, cuando el desempeño es pobre, el usuario puede ver si se debe a que el DNS o el servidor no están respondiendo, o simplemente hay congestión en la Red durante la transmisión de la página.

El diseño del URL es abierto en el sentido en que los navegadores pueden usar con facilidad varios protocolos para llegar a distintos tipos de recursos. De hecho, se han definido localizadores URL para otros protocolos. En la figura 7-19 se muestra una lista de las formas simplificadas de los URL más comunes.

Ahora analicemos la lista de forma breve. El protocolo *http* es el lenguaje nativo de la web, al que hablan los servidores web. Lo examinaremos con más detalle más adelante en esta sección.

El protocolo *ftp* se utiliza para acceder a los archivos mediante FTP, el protocolo de transferencia de archivos de Internet. FTP ya existía antes de la web; ha estado en uso por más de tres décadas. La web facilita el proceso para obtener archivos colocados en muchos servidores FTP en todo el mundo al proveer

Nombre	Se usa para	Ejemplo
http	Hipertexto (HTML).	http://www.ee.uwa.edu/~rob/
https	Hipertexto con seguridad.	https://www.bank.com/ accounts/
ftp	FTP.	ftp://ftp.cs.vu.nl/pub/minix/README
file	Archivo local.	file:///usr/suzanne/prog.c
mailto	Enviar correo electrónico.	mailto:JohnUsuario@acm.org
rtsp	Medios de flujo continuo.	rtsp://youtube.com/montypython.mpg
sip	Llamadas multimedia.	sip:eva@adversario.com
about	Información del navegador.	about:plugins

Figura 7-19. Algunos esquemas comunes de URL.

una interfaz simple en la que se puede hacer clic, en vez de una interfaz de línea de comandos. Este acceso mejorado a la información es una de las razones del crecimiento espectacular de la web.

Es posible acceder a un archivo local como una página web mediante el uso del protocolo *file*, o dicho en forma más simple, con sólo nombrarlo. Para usar este método no hay que tener un servidor. Desde luego que sólo funciona para los archivos locales, no para los remotos.

El protocolo *mailto* en realidad no tiene la habilidad de obtener páginas web, pero de todas formas es útil, ya que permite a los usuarios enviar correo electrónico desde un navegador web. La mayoría de los navegadores responderán cuando el usuario siga un vínculo *mailto* e iniciarán el agente de correo de ese usuario para redactar un mensaje con el campo de dirección llenado de antemano.

Los protocolos *rtsp* y *sip* son para establecer sesiones de medios de flujo continuo, y llamadas de audio y video.

Por último, el protocolo *about* es una convención que provee información sobre el navegador. Por ejemplo, si el usuario sigue el vínculo *about:plugins*, la mayoría de los navegadores mostrarán una página que lista los tipos MIME que pueden manejar, con las extensiones del navegador conocidas como complementos o *plug-ins*.

En resumen, los URL se diseñaron no sólo para permitir a los usuarios navegar en la web, sino también para ejecutar protocolos antiguos como FTP y el correo electrónico, así como los protocolos más recientes para audio y video, y para proveer un acceso conveniente a los archivos locales y la información del navegador. Esta metodología hace que todos los programas de interfaz de usuario especializados para esos otros servicios sean innecesarios, e integra casi todo el acceso a Internet en un solo programa: el navegador web. Si no fuera por el hecho de que un físico inglés que trabajaba en un laboratorio de investigación en Suiza concibió esta idea, podría pasar fácilmente como un plan soñado por el Departamento de Publicidad de una empresa de software.

A pesar de todas estas agradables propiedades, el uso creciente de la web ha descubierto una debilidad inherente en el esquema de URL. Un URL apunta a un host específico, pero algunas veces es conveniente hacer referencia a una página sin necesidad de decir al mismo tiempo en dónde está. Por ejemplo, en las páginas con muchas referencias es conveniente tener varias copias en ubicaciones distantes y separadas, para reducir el tráfico de red. No hay manera de decir: “Quiero la página *xyz*, pero no me importa de dónde la obtengas”.

Para resolver este tipo de problema, los URL se generalizaron en **URI (Identificadores Uniformes de Recursos)**, del inglés *Uniform Resource Identifiers*). Algunos URI indican cómo localizar un recurso. Éstos son los URL. Otros URI indican el nombre de un recurso, pero no en dónde encontrarlo. Estos URI se co-

nocen como **URN (Nombres Uniformes de Recursos)**, del inglés *Uniform Resource Names*). Las reglas para escribir nombres URI se proporcionan en el RFC 3986, mientras que la IANA rastrea los distintos esquemas URI en uso. Hay muchos tipos de URI, además de los esquemas que se listan en la figura 7-19, pero éstos son los que dominan la web según su uso actual.

Tipos MIME

Para desplegar la nueva (o cualquier) página, el navegador tiene que comprender su formato. Para que todos los navegadores puedan entender todas las páginas, éstas deben escribirse en un lenguaje común, conocido como HTML, el estándar de la web (por ahora). Analizaremos con detalle este lenguaje más adelante.

Aunque un navegador es en esencia un intérprete de HTML, la mayoría tiene muchos botones y características para facilitar el proceso de navegación; entre ellos un botón para regresar a la página anterior y otro para avanzar a la siguiente (que sólo se pueden accionar después de que el usuario haya abierto por lo menos un par de ellas), y otro para ir a la página de inicio establecida por el usuario. La mayoría de los navegadores tienen un botón o elemento de menú para establecer un favorito en una página dada, y otro para mostrar la lista de favoritos, de modo que sea posible volver a visitar esos sitios con sólo unos cuantos clics del ratón.

Como indica nuestro ejemplo, las páginas HTML pueden contener elementos de contenido complejos y no sólo texto e hipertexto. Para una generalidad adicional, no todas las páginas necesitan contener HTML. Una página puede consistir de un video en formato MPEG, un documento en formato PDF, una fotografía en formato JPEG, una canción en formato MP3 o cualquiera de cientos de otros tipos de archivos. Como las páginas HTML estándar pueden vincular a cualquiera de estos elementos, el navegador tiene un problema cuando llega a una página que no sabe cómo interpretar.

En vez de hacer los navegadores cada vez más grandes al construir intérpretes para una colección extensa de tipos de archivos que aumenta con rapidez, la mayoría de los navegadores han elegido una solución más general. Cuando un servidor devuelve una página, también devuelve cierta información adicional sobre ella. Esta información incluye el tipo MIME de la página (vea la figura 7-13). Las páginas de tipo *text/html* sólo se despliegan directamente, al igual que las páginas en unos cuantos tipos integrados más. Si el tipo MIME no es de los integrados, el navegador consulta su tabla de tipos MIME para determinar cómo desplegar la página. Esta tabla asocia los tipos MIME con los visores.

Existen dos posibilidades: complementos (*plug-ins*) y aplicaciones auxiliares. Un complemento es un módulo de código de terceros que se instala como una extensión para el navegador, como se muestra en la figura 7-20(a). Algunos ejemplos comunes son los complementos para PDF, Flash y Quicktime, de modo que se puedan desplegar los documentos, además de reproducir audio y video. Como los complementos se ejecutan dentro del navegador, tienen acceso a la página actual y pueden modificar su apariencia.

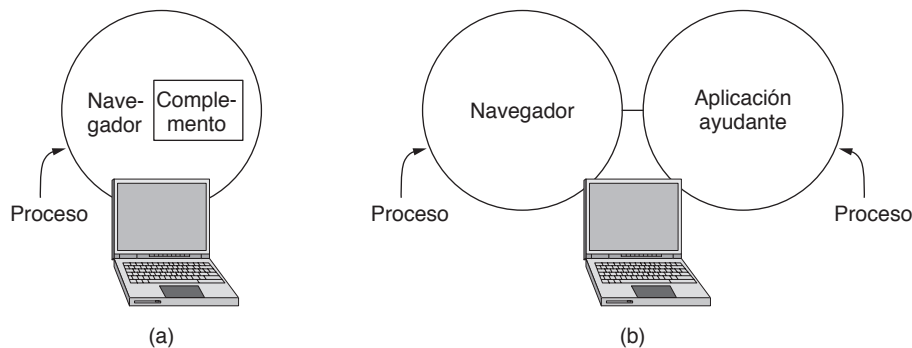


Figura 7-20. (a) Complemento del navegador. (b) Una aplicación ayudante.

Cada navegador tiene un conjunto de procedimientos que todos los complementos deben implementar para que el navegador pueda llamarlos. Por ejemplo, por lo general hay un procedimiento que el código base del navegador invoca para proveer al complemento los datos a visualizar. Este conjunto de procedimientos es la interfaz del complemento; es específica para cada navegador.

Además, el navegador crea un conjunto de sus propios procedimientos disponibles para el complemento, para que pueda proveer servicios a otros complementos. Los procedimientos comunes en la interfaz del navegador son para asignar y liberar memoria, desplegar un mensaje en la línea de estado del navegador y consultar a éste sobre los parámetros.

Antes de poder usar un complemento, hay que instalarlo. El procedimiento usual de instalación es que el usuario vaya al sitio web del complemento y descargue un archivo de instalación. Al ejecutar este archivo se desempaca el complemento y se hacen las llamadas apropiadas para registrar el tipo MIME del complemento con el navegador, además de asociarlo con el complemento. Por lo general los navegadores vienen precargados con los complementos populares.

La otra forma de extender un navegador es utilizar una **aplicación ayudante**, la cual es un programa completo que se ejecuta como un proceso separado y se ilustra en la figura 7-20(b). Como el ayudante es un programa separado, la interfaz se mantiene a distancia del navegador. Por lo general sólo acepta el nombre de un archivo de trabajo en el que se almacena el contenido, abre ese archivo y muestra el contenido en pantalla. Por lo general, los ayudantes son programas extensos que existen de manera independiente del navegador; por ejemplo, Microsoft Word o PowerPoint.

Muchas aplicaciones ayudantes usan el tipo MIME *application*. Como consecuencia se ha definido un número considerable de subtipos para que los utilicen; por ejemplo, *application/vnd.ms-powerpoint* para los archivos de PowerPoint. El término *vnd* indica formatos específicos de cada distribuidor. De esta manera, un URL puede apuntar directamente a un archivo de PowerPoint y, cuando el usuario haga clic en él, se iniciará ese programa de manera automática y se le entregará el contenido a desplegar. Las aplicaciones ayudantes no están restringidas al uso del tipo MIME *application*. Por ejemplo, Adobe Photoshop usa *image/x-photoshop*.

Asimismo, los navegadores se pueden configurar para manejar una cantidad prácticamente ilimitada de tipos de documentos sin necesidad de hacer cambios en ellos. Los servidores web modernos se configuran por lo general con cientos de combinaciones de tipo/subtipo y se agregan nuevas cada vez que se instala un nuevo programa.

Una posible fuente de conflictos es el hecho de que hay varios complementos y aplicaciones ayudantes disponibles para algunos subtipos, como *video/mpeg*. Lo que ocurre es que el último en registrarse sobrescribe la asociación existente con el tipo MIME y captura el tipo para sí mismo. Esto quiere decir que si se instala un nuevo programa, tal vez cambie la forma en que un navegador maneja los tipos existentes.

Los navegadores también pueden abrir archivos locales, si no hay una red a la vista, en vez de obtenerlos de servidores web remotos. Sin embargo, el navegador necesita alguna forma de determinar el tipo MIME del archivo. El método estándar es que el sistema operativo asocie una extensión de archivo con un tipo MIME. En una configuración típica, al abrir *foo.pdf* se abrirá en el navegador mediante el uso de un complemento *application/pdf* y al abrir *bar.doc* se abrirá en Word como la aplicación ayudante *application/msword*.

Aquí también pueden surgir conflictos, ya que muchos programas están dispuestos a (mejor dicho, deseosos de) manejar, por ejemplo, los archivos mpg. Durante la instalación, los programas destinados para usuarios sofisticados muestran con frecuencia casillas de verificación para los tipos MIME y las extensiones que están preparados a manejar, para permitir al usuario que seleccione los apropiados y de esta forma no sobrescriba las asociaciones existentes por accidente. Los programas orientados al mercado del consumidor suponen que el usuario no tiene idea de lo que es un tipo MIME, por lo que simplemente agarran todo lo que pueden sin importar lo que hayan hecho los programas que se instalaron antes.

La habilidad de extender el navegador con una gran cantidad de tipos nuevos es conveniente, pero también puede provocar problemas. Cuando un navegador en una PC con Windows obtiene un archivo con la extensión *exe*, se da cuenta de que este archivo es un programa ejecutable y, por lo tanto, no tiene ayudante. La acción obvia es ejecutar el programa. Sin embargo, esto podría ser un enorme hueco de seguridad. Todo lo que tiene que hacer un sitio web malicioso es producir una página web con imágenes de, por decir, estrellas de cine o astros del deporte, todas ellas con vínculos a un virus. Así, un solo clic en una de las imágenes provoca que se descargue y ejecute un programa ejecutable desconocido y potencialmente hostil en la máquina del usuario. Para prevenir invitados no deseados como éste, Firefox y otros navegadores vienen configurados para tener cuidado al ejecutar programas desconocidos de manera automática, pero no todos los usuarios comprenden qué opciones son seguras en vez de convenientes.

El lado del servidor

Ya vimos los detalles sobre el lado del cliente. Ahora demos un vistazo al lado del servidor. Como vimos antes, cuando el usuario escribe un URL o hace clic en una línea de hipertexto, el navegador analiza el URL e interpreta la parte entre *http://* y la siguiente barra diagonal como un nombre DNS que debe buscar. Equipado con la dirección IP del servidor, el navegador establece una conexión TCP con el puerto 80 de ese servidor. Después envía un comando que contiene el resto del URL, que es la ruta a la página en ese servidor. A continuación, el servidor devuelve la página para que el navegador la despliegue en pantalla.

A primera instancia, un servidor web simple es similar al servidor de la figura 6-6; el cual recibe el nombre de un archivo que debe buscar y devolver a través de la red. En ambos casos, los pasos que el servidor realiza en su ciclo principal son:

1. Aceptar una conexión TCP de un cliente (un navegador).
2. Obtener la ruta a la página, que viene siendo el nombre del archivo solicitado.
3. Obtener el archivo (del disco).
4. Enviar el contenido del archivo al cliente.
5. Liberar la conexión TCP.

Los servidores web modernos tienen más características, pero en esencia esto es lo que hace un servidor web para el caso simple del contenido que se encuentra en un archivo. En cuanto al contenido dinámico, el tercer paso se puede reemplazar por la ejecución de un programa (que se determina con base en la ruta) que devuelve el contenido.

Sin embargo, los servidores web se implementan con un diseño diferente para atender muchas solicitudes por segundo. Un problema con el diseño simple es que el proceso para acceder a los archivos es comúnmente el “cuello de botella”. Las lecturas de disco son muy lentas en comparación con la ejecución de un programa; además los mismos archivos se pueden leer repetidas veces del disco mediante el uso de las llamadas del sistema operativo. Otro problema es que sólo se procesa una solicitud a la vez. El archivo puede ser extenso y bloqueará a las otras solicitudes mientras se esté transfiriendo.

Una mejora evidente (que utilizan todos los servidores web) es mantener una caché en memoria de los n archivos leídos más recientes, o un cierto número de gigabytes de contenido. Así, antes de ir al disco para obtener un archivo, el servidor revisa la caché. Si el archivo está ahí, puede servirse directamente de la memoria, con lo cual se elimina el acceso al disco. Aunque un uso efectivo de la caché requiere de una gran cantidad de memoria principal y cierto tiempo de procesamiento adicional para verificar la caché y administrar su contenido, el ahorro en tiempo justifica casi siempre la sobrecarga y los gastos implícitos.

Para lidiar con el problema de atender una sola solicitud a la vez, una estrategia es hacer al servidor **multihilos**. En un diseño, el servidor consiste en un módulo de *front-end* que acepta todas las solicitudes

entrantes y k módulos de procesamiento, como se muestra en la figura 7-21. Los $k + 1$ hilos pertenecen al mismo proceso, por lo que todos los módulos de procesamiento tienen acceso a la caché dentro del espacio de direcciones del proceso. Cuando llega una solicitud, el *front-end* la acepta y construye un registro corto para describirla. Después entrega el registro a uno de los módulos de procesamiento.

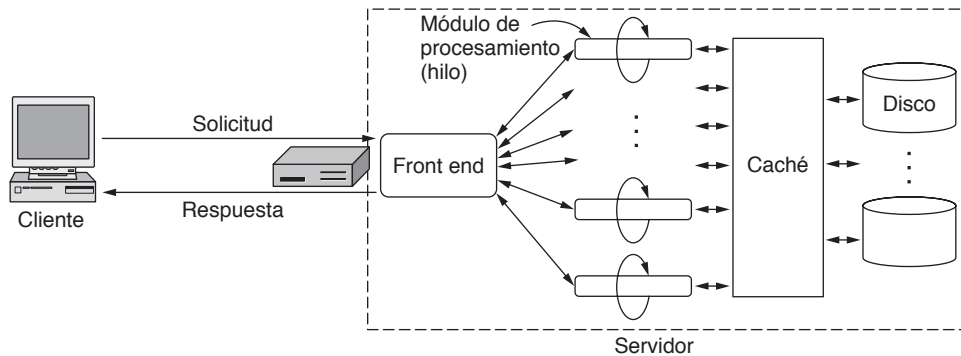


Figura 7-21. Un servidor web multihilos con un *front-end* y módulos de procesamiento.

El módulo de procesamiento primero verifica la caché para ver si el archivo solicitado está ahí. De ser así, actualiza el registro para incluir un apuntador al archivo en el registro. Si no está el módulo de procesamiento inicia una operación de disco para leerlo y colocarlo en la caché (es posible que descarte algunos otros archivos en caché para hacerle espacio). Cuando el archivo llega del disco, se coloca en la caché y también se regresa al cliente.

La ventaja de este esquema es que mientras uno o más módulos de procesamiento están bloqueados esperando a que termine una operación del disco o de red (y, por lo tanto, no consumen tiempo del CPU), otros módulos pueden estar trabajando activamente en otras solicitudes. Con k módulos de procesamiento, la tasa de transferencia puede ser hasta k veces más alta que con un servidor de un solo hilo. Por supuesto que, cuando el disco o la red son el factor limitante, es necesario tener múltiples discos o una red más veloz con el fin de obtener una mejora real en comparación con el modelo de un solo hilo.

Los modernos servidores web hacen más que sólo aceptar nombres de rutas y regresar archivos. De hecho, el procesamiento real de cada solicitud puede ser muy complicado. Por esta razón, en muchos servidores cada módulo de procesamiento realiza una serie de pasos. El *front-end* pasa cada solicitud entrante al primer módulo disponible, que a su vez la atiende mediante el uso de algún subconjunto de los siguientes pasos, dependiendo de los que sean necesarios para esa solicitud en particular. Estos pasos ocurren después de haber establecido la conexión TCP y algún mecanismo de transporte seguro (como SSL/TLS, que describiremos en el capítulo 8).

1. Resuelve el nombre de la página web solicitada.
2. Realiza control de acceso en la página web.
3. Verifica la caché.
4. Obtiene del disco la página solicitada o ejecuta un programa para construirla.
5. Determina el resto de la respuesta (por ejemplo, el tipo MIME a enviar).
6. Regresa la respuesta al cliente.
7. Realiza una entrada en el registro del servidor.

El paso 1 es necesario porque la solicitud entrante tal vez no contenga el nombre verdadero del archivo o programa como una cadena literal. Tal vez contenga accesos directos integrados que haya que traducir.

Como un ejemplo simple, el URL *http://www.cs.vu.nl/* tiene un nombre de archivo vacío. Hay que expandirlo a cierto nombre de archivo predeterminado, que por lo general es *index.html*. Otra regla común es asociar *~usuario/* al directorio web del *usuario*. Estas reglas se pueden usar juntas. Así, la página de inicio de uno de los autores (AST) se puede visitar en

http://www.cs.vu.nl/~ast/

aun cuando el verdadero nombre de archivo es *index.html* en cierto directorio predeterminado.

Además, los navegadores modernos pueden especificar información de configuración, como el software del navegador y el lenguaje predeterminado del usuario (por ejemplo, italiano o inglés). Esto hace posible que el servidor seleccione una página web con pequeñas imágenes para un dispositivo móvil y en el lenguaje preferido, si está disponible. En general, la expansión de nombres no es tan trivial como podría parecer a simple vista, debido a una variedad de convenciones sobre cómo asignar rutas al directorio de archivos y a los programas.

El paso 2 consiste en verificar si se cumple alguna restricción de acceso asociada con la página. No todas las páginas están disponibles para el público en general. Determinar cuándo un cliente puede obtener una página, puede depender de la identidad del cliente (por ejemplo, con base en los nombres de usuario y contraseñas) o de la ubicación del cliente en el DNS o espacio IP. Por ejemplo, una página puede estar restringida para los usuarios dentro de una empresa. La forma en que se logra esto depende del diseño del servidor. Es decir, para el popular servidor Apache la convención es colocar un archivo llamado *.htaccess* que lista las restricciones de acceso en el directorio en donde se encuentra la página restringida.

Los pasos 3 y 4 tratan sobre cómo obtener la página. El hecho de que se pueda obtener o no de la caché depende de las reglas de procesamiento. Por ejemplo, las páginas que se crean mediante la ejecución de programas no siempre se pueden poner en la caché, puesto que podrían producir un resultado distinto cada vez que se ejecuten. Incluso los archivos deberían verificarse de vez en cuando para ver si su contenido ha cambiado, de modo que se pueda eliminar el contenido antiguo de la caché. Si la página requiere que se ejecute un programa, existe también el problema de establecer los parámetros del programa o la entrada. Estos datos provienen de la ruta o de otras partes de la solicitud.

El paso 5 consiste en determinar otras partes de la respuesta que acompañan al contenido de la página. El tipo MIME es un ejemplo. Puede provenir de la extensión del archivo, de las primeras palabras archivo o la salida del programa, de un archivo de configuración y de otras posibles fuentes.

El paso 6 consiste en regresar la página a través de la red. Para incrementar el desempeño, un cliente y un servidor pueden usar una sola conexión TCP para obtener varias páginas. Esta reutilización significa que se requiere cierta lógica para asociar una solicitud a una conexión compartida y devolver cada respuesta de modo que esté asociada con la solicitud correcta.

El paso 7 realiza una entrada en el registro del sistema para fines administrativos, además de mantener cualquier otra estadística importante. Dichos registros se pueden usar después para extraer información valiosa sobre el comportamiento del usuario; por ejemplo, el orden en el que las personas acceden a las páginas.

Cookies

Para navegar en la web como lo hemos descrito hasta ahora, es necesario obtener una serie de páginas independientes. No existe el concepto de un inicio de sesión. El navegador envía una solicitud a un servidor y recibe un archivo. Después el servidor se olvida de haber siquiera visto a ese cliente específico.

Este modelo es perfectamente adecuado para recuperar documentos disponibles al público, y funcionaba bien durante los primeros días después de que se creó la web. Sin embargo, no es apto para regresar distintas páginas a distintos usuarios dependiendo de lo que ya hayan realizado con el servidor. Este comportamiento es necesario para muchas interacciones continuas con los sitios web. Por ejemplo, algunos sitios web

(como los periódicos) requieren que los clientes se registren (y tal vez que paguen una mensualidad) para usarlos. Aquí surge la pregunta sobre cómo pueden los servidores diferenciar las solicitudes de los usuarios previamente registrados de las solicitudes de todos los demás. Un segundo ejemplo proviene del comercio electrónico. Si un usuario navega por una tienda electrónica, enviando artículos a su carrito de compras virtual de vez en cuando, ¿cómo lleva el servidor la cuenta del contenido del carrito? Un tercer ejemplo es la personalización de portales web como Yahoo! Los usuarios pueden establecer una página inicial personalizada detallada con sólo la información que desean (por ejemplo, sus acciones y sus equipos deportivos favoritos), pero ¿cómo puede el servidor mostrar la página correcta si no sabe quién es el usuario?

En primera instancia podríamos pensar que los servidores serían capaces de rastrear a los usuarios si observaran sus direcciones IP. Sin embargo, esta idea no funciona. Muchos usuarios comparten computadoras, en especial en el hogar, y la dirección IP simplemente identifica a la computadora, no al usuario. Peor aún, muchas empresas usan NAT, de modo que los paquetes portan la misma dirección IP para todos los usuarios. Es decir, todas las computadoras detrás de la caja NAT se ven igual para el servidor. Y muchos ISP asignan direcciones IP a los clientes mediante DHCP. Las direcciones IP cambian con el tiempo, por lo que para un servidor, tal vez usted, podría de repente parecerse a su vecino. Por todas estas razones, el servidor no puede usar direcciones IP para rastrear a los usuarios.

Para resolver este problema se utiliza un mecanismo que se ha criticado muchas veces: las **cookies**. El nombre se deriva de la antigua jerga de los programadores, en la que un programa llama a un procedimiento y recibe algo que tal vez necesite presentar después para realizar cierto trabajo. En este sentido, podemos considerar a un descriptor de archivos de UNIX o a un manejador de objeto de Windows como una cookie. Las cookies se implementaron por primera vez en el navegador Netscape en 1994, y ahora se especifican en el RFC 2109.

Cuando un cliente solicita una página web, el servidor puede proveer información adicional en forma de una cookie, junto con la página solicitada. La cookie es una cadena con nombre muy pequeña (a lo más de 4 KB) que el servidor puede asociar con un navegador. Esta asociación no es lo mismo que un usuario, pero es mucho más parecida y útil que una dirección IP. Los navegadores almacenan las cookies ofrecidas durante un intervalo, por lo general en un directorio de cookies en el disco del cliente, de modo que persistan entre una invocación y otra del navegador, a menos que el usuario haya deshabilitado las cookies. Éstas son sólo cadenas, no programas ejecutables. En principio, una cookie podría contener un virus, pero ya que se tratan como datos, no hay una manera oficial de que el virus se pueda ejecutar y hacer daño. No obstante, siempre es posible que algún hacker explote un error del navegador para provocar la activación.

Una cookie puede tener hasta cinco campos, como se muestra en la figura 7-22. El *Dominio* indica de dónde viene la cookie. Se supone que los navegadores verifican que los servidores no mientan acerca de su dominio ya que cada uno puede almacenar hasta 20 cookies por cliente. La *Ruta* es una trayectoria en la estructura del directorio del servidor que identifica qué partes del árbol de archivos del servidor pueden utilizar la cookie. Por lo general es (/), lo que significa el árbol completo.

Dominio	Ruta	Contenido	Expira	Seguro
toms-casino.com	/	CustomerID=297793521	15-10-10 17:00	Sí
jills-store.com	/	Cart=1-00501;1-07031;2-13721	11-1-11 14:22	No
aportal.com	/	Prefs= Stk:CSCO+ORCL;Spt:Jets	31-12-20 23:59	No
sneaky.com	/	UserID=4627239101	31-12-19 23:59	No

Figura 7-22. Algunos ejemplos de cookies.

El campo *Contenido* toma la forma *nombre = valor*. Tanto *nombre* como *valor* pueden ser lo que el servidor desee. Este campo es donde se almacena el contenido de la cookie.

El campo *Expira* especifica cuándo caduca la cookie. Si este campo está ausente, el navegador la descarta al salir. Tal cookie se conoce como cookie **no persistente**. Si se proporciona una hora y una fecha, se dice que la cookie es **persistente** y se mantiene hasta que expira. Las horas y fechas de expiración se dan en el Tiempo del Meridiano de Greenwich. Para eliminar una cookie del disco duro de un cliente, un servidor simplemente la envía de nuevo, pero con una fecha pasada.

Por último, se puede establecer el campo *Seguro* para indicar que el navegador sólo puede regresar la cookie a un servidor mediante un transporte seguro, es decir, SSL/TLS (que describiremos en el capítulo 8). Esta característica se utiliza para comercio electrónico, actividades bancarias y otras aplicaciones seguras.

Ya vimos cómo se adquieren las cookies pero, ¿cómo se utilizan? Justo antes de que un navegador solicite una página a un sitio web, verifica su directorio de cookies para ver si el dominio al que está solicitando la página ya colocó alguna cookie. De ser así, todas las cookies colocadas por ese dominio, y sólo ese dominio, se incluyen en el mensaje de solicitud. Cuando el servidor las obtiene, puede interpretarlas de la forma que desee.

Examinemos algunos usos posibles para las cookies. En la figura 7-22, el sitio *toms-casino.com* estableció la primera cookie que se utiliza para identificar al cliente. Cuando éste regresa la siguiente semana para despilfarrar más dinero, el navegador envía la cookie de forma que el servidor sepa quién es. Una vez que el servidor cuenta con el ID del cliente, puede buscar el registro de éste en una base de datos y utilizar esta información para construir una página web apropiada para desplegar en pantalla. Dependiendo de los hábitos conocidos de apuestas del cliente, esta página podría consistir en una mano de póquer, un listado de las carreras de caballos del día o una máquina tragamonedas.

La segunda cookie proviene de *jills-store.com*. El escenario aquí es un cliente que está vagando por una tienda en busca de cosas buenas que comprar. Cuando dicho cliente encuentra una ganga y hace clic en ella, el servidor la agrega a su carrito de compras (que se mantiene en el servidor) y también crea una cookie que contiene el código del producto y envía la cookie de regreso al cliente. Conforme el cliente vaga por la tienda y hace clic en nuevas páginas, la cookie se regresa al servidor cada vez que se solicita una página nueva. Conforme se acumulan más compras, el servidor las agrega a la cookie. Por último, cuando el cliente hace clic en PASAR A LA CAJA, la cookie, que ahora contiene la lista completa de compras, se envía junto con la solicitud. De esta forma el servidor sabe con exactitud lo que el cliente desea comprar.

La tercera cookie es para un portal web. Cuando el cliente hace clic en un vínculo que lo lleva al portal, el navegador envía la cookie. Ésta le indica al portal que construya una página que contenga los precios de las acciones de Cisco y Oracle, así como los resultados de fútbol de los Jets de Nueva York. Puesto que una cookie puede tener un tamaño de hasta 4 KB, hay bastante espacio para incluir preferencias más detalladas, como encabezados de los periódicos, el clima local, ofertas especiales, etcétera.

Un uso más controversial de las cookies es el de rastrear el comportamiento en línea de los usuarios. Esto permite a los operadores de sitios web comprender cómo los usuarios navegan en sus sitios; además los anunciantes crean perfiles de los anuncios o sitios que ha visto un usuario específico. La controversia es que los usuarios por lo general no están conscientes de que se está rastreando su actividad, incluso con perfiles detallados y a través de sitios web sin relación aparente. Sin embargo, el **rastreo web** es un gran negocio. DoubleClick, que provee y rastrea anuncios, está clasificado entre los 100 sitios web más ocupados del mundo por la compañía de monitoreo Alexa. Google Analytics, que rastrea el uso de los sitios para los operadores, se utiliza en más de la mitad de los 100 000 sitios más ocupados en la web.

Es fácil para un servidor rastrear la actividad de los usuarios mediante las cookies. Suponga que un servidor desea mantener el registro de cuántos visitantes únicos ha tenido y cuántas páginas vio un visitante antes de salir del sitio. Cuando llega la primera solicitud no hay cookie que la acompañe, por lo que

el servidor regresa una cookie que contiene $Counter = 1$. Las visitas subsiguientes a las páginas en ese sitio regresarán la cookie al servidor. Cada vez se incrementa el contador y se regresa al cliente. Al llevar un registro de los contadores, el servidor puede ver cuántas personas salieron del sitio después de ver la primera página, cuántas vieron dos páginas y así sucesivamente.

Rastrear el comportamiento de navegación de los usuarios entre un sitio y otro es sólo un poco más complicado. Funciona de la siguiente manera. Una agencia de publicidad, digamos, Sneaky Ads, contacta sitios web importantes y coloca en sus páginas anuncios de los productos de sus clientes, por lo cual paga a los dueños de los sitios una cuota. En lugar de dar a los sitios un archivo GIF para que lo coloquen en cada página, les proporciona un URL para dicho propósito. Cada URL que dicha agencia maneja contiene un número único en la ruta, como

<http://www.sneaky.com/382674902342.gif>

Cuando un usuario visita por primera vez una página, P , que contiene un anuncio de éstos, el navegador obtiene el archivo HTML. A continuación el navegador inspecciona el archivo HTML y ve el vínculo al archivo de imagen en www.sneaky.com, por lo que envía ahí un mensaje en el que solicita la imagen. Se regresa un archivo GIF que contiene un anuncio, junto con una cookie que contiene un ID de usuario único, 4627239101 en la figura 7-22. Sneaky registra el hecho de que el usuario con este ID visitó la página P . Esto es fácil de hacer puesto que se hace referencia a la ruta solicitada ([382674902342.gif](http://www.sneaky.com/382674902342.gif)) sólo en la página P . Por supuesto, el anuncio real puede aparecer en miles de páginas, pero cada vez con un nombre diferente. Sneaky probablemente cobre un par de centavos al fabricante del producto cada vez que envíe el anuncio.

Más tarde, cuando el usuario visita otra página web que contenga cualquiera de los anuncios de Sneaky, el navegador primero obtiene el archivo HTML del servidor. Después ve el vínculo a, digamos, <http://www.sneaky.com/193654919923.gif> y solicita ese archivo. Puesto que ya tiene una cookie del dominio [sneaky.com](http://www.sneaky.com), el navegador incluye una cookie de Sneaky que contiene el ID del usuario. Sneaky ahora conoce una segunda página que el usuario ha visitado.

A su debido tiempo, Sneaky puede construir un perfil detallado de los hábitos de navegación del usuario, aunque éste nunca haya hecho clic en ninguno de los anuncios. Por supuesto, aún no tiene el nombre del usuario (aunque tiene su dirección IP, lo cual debe ser suficiente para deducir el nombre a partir de otras bases de datos). Pero si el usuario alguna vez proporciona su nombre a algún sitio que coopere con Sneaky, estará disponible un perfil completo junto con un nombre para venderlo a quien desee comprarlo. La venta de esta información puede ser lo bastante rentable como para que Sneaky coloque más anuncios en más sitios web y, por lo tanto, para que recolecte más información.

Y si Sneaky desea ser “supersneaky”, el anuncio no necesita ser el clásico anuncio de pancarta (*banner*). Un “anuncio” que conste de un solo píxel en el color de fondo (lo que lo hace invisible) tiene exactamente el mismo efecto que un anuncio de pancarta: requiere que el navegador obtenga la imagen GIF de 1×1 píxeles y le envíe todas las cookies que se originan en el dominio del píxel.

Las cookies se han convertido en un punto central de debate en relación con la privacidad en línea, debido al comportamiento de rastreo antes mostrado. La parte más insidiosa de todo es que muchos usuarios están totalmente inconscientes de esta recolección de información, e incluso pueden pensar que están seguros debido a que no hacen clic en ninguno de los anuncios. Por esta razón, muchos consideran las cookies que rastrean a los usuarios de un sitio a otro como **spyware**. Dé un vistazo a las cookies que su navegador ya tiene almacenadas. La mayoría de los navegadores desplegarán esta información junto con las preferencias de privacidad actuales. Tal vez le sorprenda encontrar nombres, direcciones de correo electrónico o contraseñas, así como identificadores opacos. Con suerte y no encontrará números de tarjetas de crédito, pero el potencial de abuso está claro.

Para mantener algo de su privacidad, algunos usuarios configuran sus navegadores para que rechacen todas las cookies. Sin embargo, esto puede causar problemas debido a que muchos sitios web no funcio-

nan bien sin cookies. Como alternativa, la mayoría de los navegadores permiten a los usuarios bloquear las **cookies de terceros**. Una cookie de tercero proviene de un sitio distinto al de la página principal que se obtiene; por ejemplo, la cookie de *sneaky.com* que se utiliza al interactuar con la página *P* en un sitio web completamente distinto. Bloquear estas cookies ayuda a evitar el rastreo entre un sitio web y otro. También se pueden instalar extensiones del navegador para proveer un control más detallado sobre la forma en que se usan las cookies (o más bien, que no se usan). A medida que continúa el debate, muchas empresas están desarrollando políticas privadas que limitan la forma en que compartirán información para evitar el abuso. Claro está que las políticas consisten simplemente en la forma como las empresas dicen que van a manejar la información. Por ejemplo: “Podemos usar la información que recolectamos sobre usted para llevar a cabo nuestro negocio”, cuyo negocio podría ser vender la información.

7.3.2 Páginas web estáticas

La base de la web es transferir páginas web del servidor al cliente. En su forma más simple, las páginas web son estáticas. Es decir, son sólo archivos guardados en un servidor que se presentan a sí mismos de la misma forma cada vez que un cliente los obtiene y los ve. Sin embargo, el hecho de que son estáticas no significa que las páginas sean inertes en el navegador. Una página que contenga un video puede ser una página web estática.

Como dijimos antes, la lengua franca de la web, en la que se escriben la mayoría de las páginas, es HTML. Las páginas de inicio de los maestros por lo general son páginas HTML estáticas. Las páginas de inicio de las empresas son comúnmente páginas dinámicas creadas por una empresa de diseño web. En esta sección daremos un breve vistazo a las páginas HTML estáticas como base para el material posterior. Los lectores que ya estén familiarizados con el HTML pueden saltar hasta la siguiente sección, en donde describiremos el contenido dinámico y los servicios web.

HTML: Lenguaje de Marcado de HiperTexto

El **HTML (Lenguaje de Marcado de HiperTexto)**, del inglés *HyperText Markup Language*) se introdujo con la web. Permite a los usuarios producir páginas web que incluyen texto, gráficos, video, apuntadores a otras páginas web y más. HTML es un lenguaje de marcado que sirve para describir cómo se va a dar formato a los documentos. El término “marcado” proviene de la época en que los correctores de estilo realmente marcaban los documentos para indicar a la imprenta —en aquellos tiempos, un humano— qué fuentes utilizar, y cosas por el estilo. Por lo tanto, los lenguajes de marcado contienen comandos explícitos para dar formato. Por ejemplo, en HTML, **** significa **iniciar modo en negritas** y **** significa abandonar modo en negritas. LaTeX y TeX son otros ejemplos de lenguajes de marcado bien conocidos para la mayoría de los autores académicos.

La ventaja clave de un lenguaje de marcado sobre uno con marcado no explícito es que separa el contenido de la forma en que se debe presentar. Así, escribir un navegador es algo directo: el navegador simplemente tiene que entender los comandos de marcado y aplicarlos al contenido. Al integrar todos los comandos de marcado dentro de cada archivo HTML y estandarizarlos, se hace posible que cualquier navegador web lea y vuelva a dar formato a cualquier página web. Esto es crucial porque una página se pudo haber producido en una ventana de 1600×1200 con colores de 24 bits en una computadora de gama alta, pero tal vez se vaya a desplegar en una ventana de 640×320 en un teléfono móvil.

Aunque es posible escribir documentos como éstos con cualquier editor estándar de texto (y mucha gente lo hace), también es posible usar procesadores de texto o editores de HTML especiales que pueden hacer la mayoría del trabajo (pero que por lo mismo dan al usuario menos control sobre todos los detalles del resultado final).

En la figura 7-23 se muestra una página web simple escrita en HTML, y su presentación en un navegador. Una página web está constituida por un encabezado y un cuerpo, cada uno de los cuales está encerrado entre **etiquetas** (comandos de formato) `<html>` y `</html>`, aunque la mayoría de los navegadores no se quejan si faltan estas etiquetas. Como se puede ver en la figura 7-23(a), el encabezado está delimitado por las etiquetas `<head>` y `</head>`; y el cuerpo por `<body>` y `</body>`. Las cadenas dentro de

```
<html>
<head> <title> WIDGETS AMALGAMADOS, S.A. </title> </head>
<body> <h1> Bienvenidos a la página de inicio de WASA </h1>
 <br>
Estamos muy contentos de que haya elegido visitar la página de inicio de <b>
Widgets Amalgamados </b>. Esperamos que <i> usted </i> encuentre aquí toda la información que necesita.
<p> A continuación presentamos vínculos con la información sobre nuestro surtido de productos finos. Puede
ordenar en forma electrónica (por WWW), por teléfono o por correo electrónico. </p>
<hr>
<h2> Información sobre nuestros productos </h2>
<ul>
  <li> <a href="http://widget.com/productos/grandes"> Widgets grandes </a> </li>
  <li> <a href="http://widget.com/productos/chicos"> Widgets chicos </a> </li>
</ul>
<h2> Información de contacto </h2>
<ul>
  <li> Teléfono: 1-800-WIDGETS </li>
  <li> Correo electrónico: info@widgets-amalgamados.com </li>
</ul>
</body>
</html>
```

(a)

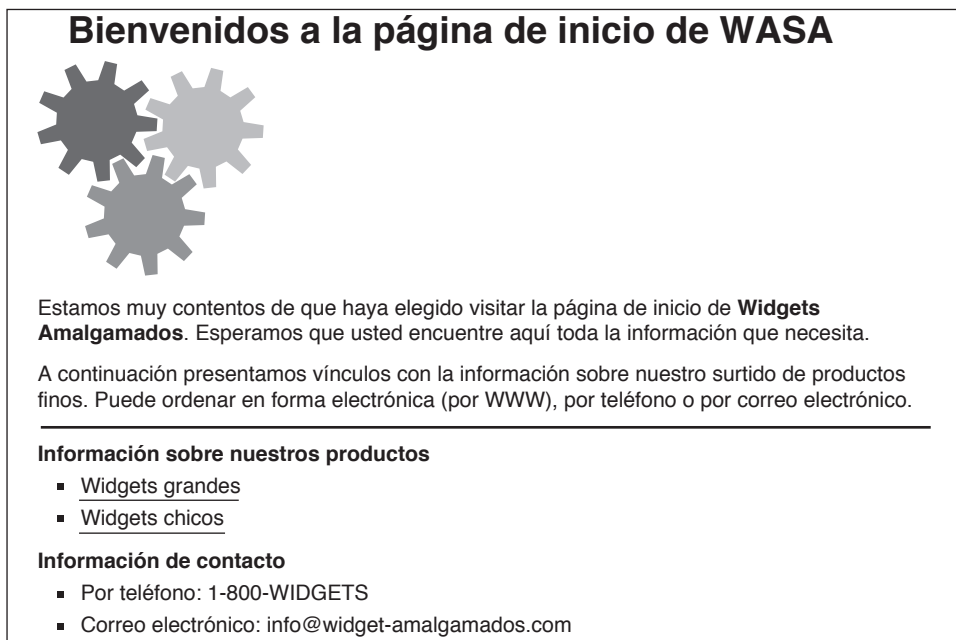


Figura 7-23. (a) El HTML para una página web de ejemplo. (b) La página con formato.

las etiquetas se llaman **directivas**. La mayoría de las etiquetas HTML tiene este formato. Es decir, usan `<algo>` para marcar el comienzo de algo, y `</algo>` para marcar su fin.

Las etiquetas se pueden escribir tanto en minúsculas como en mayúsculas. Por lo tanto, `<head>` y `<HEAD>` significan lo mismo, pero es mejor usar minúsculas por cuestión de compatibilidad. La distribución real del documento HTML no es importante. Los analizadores de HTML ignoran los espacios extra y los retornos, puesto que tienen que volver a dar formato al texto para acomodarlo en el área de visualización actual. En consecuencia, se puede agregar espacio en blanco a voluntad para hacer más legibles los documentos HTML, algo que la mayoría necesita con urgencia. Otro resultado es la imposibilidad de usar líneas en blanco para separar párrafos, puesto que simplemente se ignoran. Se requiere una etiqueta explícita.

Algunas etiquetas tienen parámetros (con nombre), llamados **atributos**. Por ejemplo, la etiqueta `<img>` de la figura 7-23 se utiliza para incluir una imagen en línea con el texto. Tiene dos atributos: `src` y `alt`. El primero proporciona el URL para la imagen. El estándar HTML no especifica qué formatos de imagen se permiten. En la práctica, todos los navegadores soportan archivos GIF y JPEG. Los navegadores son libres de soportar otros formatos, pero esta extensión es una espada de doble filo. Si un usuario está acostumbrado a un navegador que soporta, por decir, archivos TIFF, puede incluirlos en sus páginas web para después sorprenderse cuando otros navegadores simplemente ignoren todo su maravilloso arte.

El segundo atributo proporciona el texto alternativo a usar si no se puede mostrar la imagen. Para cada etiqueta, el estándar de HTML proporciona una lista de cuáles son los parámetros permitidos (si los hay) y lo que significan. Como cada parámetro tiene un nombre, el orden en el que se dan no es importante.

Técnicamente, los documentos HTML se escriben en el conjunto de caracteres Latin-1 del ISO 8859-1, pero para los usuarios cuyos teclados sólo reconocen ASCII, hay secuencias de escape para los caracteres especiales, como `è`. La lista de caracteres especiales se proporciona en el estándar. Todos comienzan con un signo `&` y terminan con un punto y coma. Por ejemplo, ` ` produce un espacio, ``` produce `è` y `´` produce `é`. Puesto que `<`, `>`, y `&` tienen significados especiales, se pueden expresar sólo con sus secuencias de escape, `<`, `>` y `&`, respectivamente.

El elemento principal del encabezado es el título, delimitado por `<title>` y `</title>`. También pueden estar presentes ciertos tipos de metainformación, aunque ninguno está presente en nuestro ejemplo. El título mismo no se despliega en la página. Algunos navegadores lo usan para rotular la ventana de la página.

En la figura 7-23 se utilizan varios encabezados. Cada encabezado se genera mediante una etiqueta `<hn>`, donde n es un dígito en el intervalo de 1 a 6. Por lo tanto, `<h1>` es el encabezado más importante y `<h6>` es el menos importante. Es responsabilidad del navegador desplegar estos encabezados de manera adecuada en la pantalla. Por lo general, los encabezados de menor número se presentarán con una fuente más grande y gruesa. El navegador también puede seleccionar colores distintos para cada nivel de encabezado. Por lo común, los encabezados `<h1>` son grandes y en negritas, con al menos una línea en blanco arriba y abajo. En contraste, los encabezados `<h2>` tienen una fuente más pequeña y con menos espacio arriba y abajo.

Las etiquetas `<b>` e `<i>` se usan para entrar en el modo de negritas y cursivas, respectivamente. La etiqueta `<hr>` inserta de manera obligatoria un salto de línea y dibuja una línea horizontal a través de la pantalla.

La etiqueta `<p>` inicia un párrafo. Para desplegarlo, el navegador podría insertar una línea en blanco y algo de sangría, por ejemplo. Como dato interesante, los programadores HTML perezosos omiten con frecuencia la etiqueta `</p>` que existe para marcar el final de un párrafo.

HTML proporciona varios mecanismos para crear listas, incluyendo las listas anidadas. Las listas desordenadas, como las que se muestran en la figura 7-23, inician con `<ul>` y `<li>` se usa para marcar el inicio de los elementos. También existe una etiqueta `<ol>` para iniciar una lista ordenada. Los elementos

individuales en las listas desordenadas aparecen por lo general con viñetas (•) al principio. El navegador se encarga de enumerar los elementos en las listas ordenadas.

Por último, llegamos a los hipervínculos. En la figura 7-23 podemos ver ejemplos de éstos, que utilizan las etiquetas `<a>` (ancla) y `</a>`. La etiqueta `<a>` tiene varios parámetros, de los cuales el más importante es *href*, el URL vinculado. El texto entre `<a>` y `</a>` se despliega en pantalla. Si se selecciona, se sigue el hipervínculo a una nueva página. También se permite vincular a otros elementos. Por ejemplo, se puede proporcionar una imagen entre `<a>` y `</a>` mediante el uso de `<img>`. En este caso, la imagen se despliega y, al hacer clic en ella, se activa el hipervínculo.

Existen muchas otras etiquetas y atributos HTML que no hemos visto en este ejemplo simple. Por ejemplo, la etiqueta `<a>` puede tomar un parámetro *nombre* para plantar un hipervínculo, lo cual le permite apuntar a la parte media de una página. Esto es útil, por ejemplo, para páginas web que empiezan con una tabla de contenido en la que se puede hacer clic. Al hacer clic en un elemento en la tabla de contenido, el usuario salta a la sección correspondiente de la misma página. Un ejemplo de una etiqueta distinta es `<br>`, la cual obliga al navegador a interrumpir una línea y empezar una nueva.

Quizá la mejor manera de comprender las etiquetas sea verlas en acción. Para ello puede elegir una página web y analizar el HTML en su navegador, para ver cómo se elaboró esta página. La mayoría de los navegadores tienen un elemento de menú llamado VER CÓDIGO FUENTE (VIEW SOURCE), o algo similar. Al seleccionar este elemento se visualiza el código HTML de la página actual, en vez de su salida con formato.

Elaboramos un bosquejo de las etiquetas que han existido desde los principios de la web. El HTML sigue evolucionando. La figura 7-24 muestra partes de las características que se han agregado con versiones sucesivas de HTML. El HTML 1.0 se refiere a la versión de HTML utilizada al momento de introducir la web. Las versiones 2.0, 3.0 y 4.0 de HTML aparecieron en rápida sucesión en el transcurso de unos cuantos años, a medida que la web se expandió drásticamente. Después de HTML 4.0, pasó un periodo de casi 10 años para que se viera clara la ruta hacia la estandarización de la siguiente versión importante: HTML 5.0. Como es una actualización importante que consolida los medios por los que los navegadores manejan el contenido rico, el esfuerzo de HTML 5.0 es continuo y no se espera que produzca un estándar sino hasta después de 2012, a lo más. No obstante los estándares, los principales navegadores ya soportan la funcionalidad de HTML 5.0.

La progresión por las versiones de HTML trata sobre agregar nuevas características que la gente deseaba pero tenía que manejar en formas no estandarizadas (como los complementos) hasta que se hicieron estándar. Por ejemplo, HTML 1.0 y HTML 2.0 no tenían tablas. Se agregaron en el HTML 3.0. Una tabla de HTML consiste en una o más filas, cada una de las cuales consiste en una o más celdas en la tabla que contienen una amplia variedad de material (por ejemplo, texto, imágenes, otras tablas). Antes de HTML 3.0, los autores que necesitaban una tabla tenían que recurrir a métodos apropiados, como incluir una imagen que mostrara la tabla.

En el HTML 4.0 se agregaron más características nuevas. Éstas incluían características de accesibilidad para los usuarios discapacitados, la incrustación de objetos (una generalización de la etiqueta `<img>` de modo que también se puedan incrustar otros objetos en las páginas), soporte para los lenguajes de secuencias de comandos (para permitir contenido dinámico), y otras cosas más.

El HTML 5.0 incluye muchas características para manejar los medios ricos en contenido que ahora se utilizan de manera rutinaria en la web. Se puede incluir video y audio en las páginas para que lo reproduzca el navegador, sin que el usuario tenga que instalar complementos. Los dibujos se pueden acumular en el navegador como gráficos vectoriales, en vez de usar formatos de imágenes de mapas de bits (como JPEG y GIF). También hay más soporte para ejecutar secuencias de comandos en los navegadores, como los hilos de cálculos en segundo plano y acceso al almacenamiento. Todas estas características ayudan a soportar páginas web que sean más parecidas a las aplicaciones tradicionales con una interfaz de usuario que a los documentos. Ésta es la dirección que está siguiendo la web.

Elemento	HTML 1.0	HTML 2.0	HTML 3.0	HTML 4.0	HTML 5.0
Hipervínculos	x	x	x	x	x
Imágenes	x	x	x	x	x
Listas	x	x	x	x	x
Mapas e imágenes activas		x	x	x	x
Formularios		x	x	x	x
Ecuaciones			x	x	x
Barras de herramientas			x	x	x
Tablas			x	x	x
Características de accesibilidad				x	x
Incrustación de objetos				x	x
Hojas de estilo				x	x
Secuencias de comandos				x	x
Video y audio					x
Gráficos vectoriales en línea					x
Representación de XML					x
Hilos en segundo plano					x
Almacenamiento del navegador					x
Lienzo de dibujo					x

Figura 7-24. Algunas diferencias entre versiones de HTML.

Entrada y formularios

Hay una importante herramienta que aún no hemos examinado: la entrada de datos. En esencia, HTML 1.0 era de un solo sentido. Los usuarios podían obtener las páginas de información de los proveedores, pero era difícil enviar información en el otro sentido. Pronto se hizo obvio que se necesitaba tráfico de dos vías para colocar pedidos de productos a través de páginas web, llenar tarjetas de registro en línea, introducir términos de búsqueda y mucho, mucho más.

Para enviar entrada del usuario al servidor (a través del navegador) se requieren dos tipos de soporte. Primero, que HTTP pueda transportar datos en esta dirección. En una sección posterior describiremos cómo hacer esto; se utiliza el método *POST*. El segundo requerimiento es presentar elementos de la interfaz de usuario que recopilen y empaqueten la entrada. En el HTML 2.0 se incluyeron **formularios** con esta funcionalidad.

Los formularios contienen cuadros o botones que permiten a los usuarios proporcionar información o tomar decisiones, y después enviar dicha información al dueño de la página. Se escriben justo igual que otras partes de HTML, como podemos ver en el ejemplo de la figura 7-25. Debemos tener en cuenta que los formularios siguen siendo contenido estático. El contenido dinámico, que analizaremos más adelante, provee formas más sofisticadas de recopilar la entrada mediante el envío de un programa cuyo comportamiento puede depender del entorno del navegador.


```

<html>
<head> <title> FORMULARIO DE PEDIDO DE CLIENTES WASA </title> </head>
<body>
<h1> Orden de compra de widgets </h1>
<form ACTION="http://widget.com/cgi-bin/pedido.cgi" method=POST>
<p> Nombre <input name="cliente" size=46> </p>
<p> Dirección <input name="direccion" size=40> </p>
<p> Ciudad <input name="city" size=20> Estado <input name="estado" size =4>
País <input name="pais" size=10> </p>
<p> Núm. tarjeta de crédito <input name="numtarj" size=10>
Expira <input name="expira" size=4>
M/C <input name="tc" type=radio value="mastercard">
VISA <input name="tc" type=radio value="visacard"> </p>
<p> Tamaño de widget Grande <input name="producto" type=radio value="costoso">
Pequeño <input name="producto" type=radio value="barato">
Enviar por mensajería rápida <input name="express" type=checkbox> </p>
<p><input type=submit value="Enviar pedido"> </p>
¡Gracias por ordenar un widget WASA, el mejor widget que el dinero puede comprar!
</form>
</body>
</html>

```

(a)

(b)

Figura 7-25. (a) El HTML para un formulario de pedido. (b) La página con formato.

Al igual que todos los formularios, éste va encerrado entre las etiquetas `<form>` y `</form>`. Los atributos de esta etiqueta indican lo que se debe hacer con los datos que se introducen, en este caso mediante el método *POST* para enviar los datos al URL especificado. El texto que no va encerrado en una etiqueta simplemente se despliega en pantalla. Todas las etiquetas usuales (por ejemplo, `<b>`) se permiten en un formulario para que el autor de la página pueda controlar su apariencia en la pantalla.

En este formulario se utilizan tres tipos de cuadros de entrada, cada uno de los cuales usa la etiqueta `<input>`. Tiene una variedad de parámetros para determinar el tamaño, naturaleza y uso del cuadro

desplegado. Los formularios más comunes son campos en blanco para aceptar texto del usuario, cuadros que se pueden activar y botones *submit* que hacen que se devuelvan los datos al servidor.

El primer tipo de cuadro de entrada es un cuadro *text* que va después del texto “Nombre”. El cuadro es de 46 caracteres de ancho y espera que el usuario escriba una cadena de texto, la cual a su vez se almacena en la variable llamada *cliente*.

En la siguiente línea del formulario se pide la dirección del usuario, de 40 caracteres de ancho. Después viene una línea que pide la ciudad, el estado y el país. Como no se utilizan etiquetas `<p>` entre estos campos, el navegador los muestra todos en una línea (en vez de mostrarlos como párrafos separados) si es que caben. En lo que respecta al navegador, el único párrafo contiene sólo seis elementos: tres cadenas de texto alternadas con tres cuadros. La siguiente línea pide el número de tarjeta de crédito y la fecha de expiración. Sólo hay que transmitir números de tarjetas de crédito a través de Internet cuando se hayan tomado las medidas de seguridad adecuadas. En el capítulo 8 hablaremos sobre esto.

Después de la fecha de expiración nos encontramos con una nueva herramienta: los botones de opción. Éstos se utilizan cuando hay que elegir entre dos o más alternativas. Aquí, el modelo intelectual es la radio de un automóvil con media docena de botones para elegir estaciones. Al hacer clic en un botón se desactivan todos los demás botones del mismo grupo. La presentación visual depende del navegador. El tamaño de *widget* también usa dos botones de opción. Los dos grupos se diferencian mediante su parámetro *nombre* y no mediante un alcance estático en el que se utiliza algo como `<radiobutton> ... </radiobutton>`.

Los parámetros *value* se utilizan para indicar qué botón de opción se oprimió. Por ejemplo, dependiendo de las opciones de tarjeta de crédito que haya elegido el usuario, se establecerá la variable *cc* con la cadena “mastercard” o la cadena “visacard”.

Después de los dos conjuntos de botones de opción, llegamos a la opción de envío, representada por un cuadro de tipo *checkbox* (casilla de verificación). Puede estar activado o desactivado. A diferencia de los botones de opción, en donde debe estar seleccionado exactamente uno del conjunto, cada cuadro de tipo *checkbox* puede estar activado o desactivado en forma independiente de los otros.

Finalmente llegamos al botón enviar (*submit*). La cadena *value* es la etiqueta del botón y se despliega en pantalla. Cuando el usuario hace clic en el botón *enviar*, el navegador empaqueta la información recolectada en una sola línea larga y la envía de vuelta al servidor, al URL que se proporciona como parte de la etiqueta `<form>`. Se utiliza una codificación simple. El signo `&` se usa para separar campos y el signo `+` se usa para representar el espacio. En nuestro formulario de ejemplo, la línea se podría ver como el contenido de la figura 7-26.

```
cliente=John+Doe&direccion=100+Main+St.&ciudad=White+Plains&
estado=NY&pais=USA&numtarj=1234567890&expira=6/14&cc=mastercard&
producto=barato&express=on
```

Figura 7-26. Una posible respuesta del navegador para el servidor, con la información escrita por el usuario.

La cadena se envía de vuelta al servidor como una línea (aquí se divide en tres líneas debido a que la página no tiene el ancho suficiente). Es responsabilidad del servidor dar sentido a esta cadena; lo más probable es que pase la información a un programa que la procese. En la siguiente sección veremos cómo se puede realizar esto.

Existen también otros tipos de entrada que no se muestran en este ejemplo simple. Dos de estos otros tipos son *password* y *textarea*. Un cuadro *password* es igual que un cuadro *text* (el tipo predeterminado que no necesita nombrarse), excepto que los caracteres no se muestran a medida que se escriben. Un cuadro *textarea* es también lo mismo que un cuadro *text*, sólo que puede contener varias líneas.

Para las listas largas en las que haya que elegir una opción, se proveen las etiquetas `<select>` y `</select>` para agrupar una lista de alternativas. A menudo esta lista se visualiza como un menú desplegable. La semántica es como la de los botones de opción, a menos que se proporcione el parámetro *multiple*, en cuyo caso la semántica es como la de las casillas de verificación.

Finalmente, hay maneras de indicar valores predeterminados o iniciales que el usuario puede modificar. Por ejemplo, si a un cuadro *text* se le asigna un campo *value*, el contenido se despliega en el formulario para que el usuario lo edite o borre.

CSS: Hojas de Estilo en Cascada

El objetivo original del HTML era especificar la *estructura* del documento, no su *apariciencia*. Por ejemplo,

```
<h1>Fotos de Débora</h1>
```

indica al navegador que debe enfatizar el encabezado, pero no dice nada sobre el tipo de letra, tamaño de punto o color. Esto queda a discreción del navegador, que conoce las propiedades de la pantalla (por ejemplo, cuántos píxeles tiene). Sin embargo, muchos diseñadores de páginas web querían un control absoluto sobre la forma en que aparecían sus páginas, por lo que se agregaron nuevas etiquetas a HTML para controlar la apariencia, como:

```
<font face="helvética" size="24" color="red">Fotos de Débora</font>
```

Además se agregaron formas de controlar el posicionamiento en la pantalla con precisión. El problema con este método es que es tedioso y produce HTML inflado que no es portable. Aunque una página se puede visualizar perfectamente en el navegador en el que se desarrolló, puede ser un completo desastre en otro navegador, en otra versión del mismo navegador o a una resolución de pantalla distinta.

Una mejor alternativa es el uso de las hojas de estilo. En los editores de texto, las hojas de estilo permiten a los autores asociar texto con un estilo lógico en vez de uno físico; por ejemplo, “párrafo inicial” en vez de “texto en cursiva”. La apariencia de cada estilo se define por separado. De esta manera, si el autor decide cambiar los párrafos iniciales de cursiva a 14 puntos en color azul, a negritas a 18 puntos en rosa impactante, todo lo que requiere es modificar una definición para convertir todo el documento.

El concepto de **CSS (Hojas de Estilo en Cascada)**, del inglés *Cascading Style Sheets*) introdujo las hojas de estilo a la web con el HTML 4.0, aunque no se empezaron a popularizar y los navegadores no ofrecieron soporte sino hasta el año 2000. CSS define un lenguaje simple para describir reglas que controlan la apariencia de contenido etiquetado. Veamos un ejemplo. Suponga que WASA desea páginas web elegantes con texto color azul marino en fuente Arial sobre un fondo blanco, y encabezados de nivel que son 100% y 50% más grandes que el texto para cada nivel, respectivamente. La definición de CSS en la figura 7-27 proporciona estas reglas.

```
body {background-color:linen; color:navy; font-family:Arial;}
h1 {font-size:200%}
h2 {font-size:150%}
```

Figura 7-27. Ejemplo de CSS.

Como vemos en la figura anterior, las definiciones de estilo pueden ser compactas. Cada línea selecciona un elemento al que aplica y proporciona los valores de las propiedades. Las propiedades de un elemento se aplican como valores predeterminados para todos los demás elementos de HTML que contiene. Así, el estilo para *body* establece el estilo para los párrafos de texto en el cuerpo. También hay

convenientes métodos abreviados para los nombres de los colores (por ejemplo, **red**). El navegador llena con valores predeterminados cualquier parámetro de estilo que no se defina. Este comportamiento hace a las definiciones en la hoja de estilo opcionales; sin ellas puede haber una presentación razonable.

Podemos colocar las hojas de estilo en un archivo HTML (mediante la etiqueta `<style>`), pero es más común colocarlas en un archivo separado y hacer referencia a ellas. Por ejemplo, podemos modificar la etiqueta `<head>` de la página WASA para hacer referencia a una hoja de estilo en el archivo *estilowasa.css*, como se muestra en la figura 7-28. El ejemplo también muestra que el tipo MIME de los archivos CSS es *text/css*.

```
<head>
<title>WIDGETS AMALGAMADOS, S.A. </title>
<link rel="stylesheet" type="text/css" href="awistyle.css" />
</head>
```

Figura 7-28. Inclusión de una hoja de estilo CSS.

Esta estrategia tiene dos ventajas. En primer lugar, permite aplicar un conjunto de estilos a muchas páginas en un sitio web. Esta organización otorga una apariencia consistente a las páginas, incluso aunque las hayan desarrollado distintos autores en distintos momentos, y permite modificar la apariencia de todo el sitio con sólo editar un solo archivo CSS y no el HTML. Podemos comparar este método con un archivo `#include` en un programa en C: al cambiar la definición de una macro ahí, se cambia en todos los archivos de programa que incluyen ese encabezado. La segunda ventaja es que los archivos de HTML que se descargan se mantienen en un tamaño pequeño. Esto se debe a que el navegador puede descargar una copia del archivo CSS para todas las páginas que lo referencien. No necesita descargar una nueva copia de las definiciones junto con cada página web.

7.3.3 Páginas web dinámicas y aplicaciones web

El modelo de página estático que hemos usado hasta ahora trata a las páginas como documentos multimedia que están vinculados de una manera conveniente. Fue un modelo adecuado en los primeros días de la web, cuando se pusieron en línea grandes cantidades de información. Hoy en día, la mayor parte de la emoción que rodea a la web está en usarla para aplicaciones y servicios. Ejemplos de ello son: comprar productos en sitios de comercio electrónico, buscar en catálogos de bibliotecas, explorar mapas, leer y enviar correo electrónico, y colaborar en documentos.

Estos nuevos usos son como el software de aplicación tradicional (por ejemplo, lectores de correo y procesadores de palabras). El detalle es que estas aplicaciones se ejecutan dentro del navegador, y los datos de usuario se almacenan en servidores en centros de datos de Internet. Utilizan protocolos web para acceder a la información a través de Internet, y el navegador para mostrar una interfaz de usuario. La ventaja de este método es que los usuarios no necesitan instalar programas de aplicación separados, y se puede acceder a los datos de usuario desde distintas computadoras, además de que el operador del servicio respalda la información. Está demostrando tener tanto éxito que rivaliza con el software de aplicación tradicional. Claro que también ayuda el hecho de que los grandes proveedores ofrezcan estas aplicaciones sin costo. Este modelo es la forma común de **computación en nube**, en la cual la computación pasa de las computadoras de escritorio individuales hacia grupos compartidos de servidores en Internet.

Para que actúen como aplicaciones, las páginas web ya no pueden ser estáticas. Es necesario el contenido dinámico. Por ejemplo, una página del catálogo de una biblioteca debe reflejar los libros disponibles en un momento dado, además de los libros que están prestados y por lo tanto no se encuentran disponibles.

Asimismo, una página útil de la Bolsa de Valores debe permitir al usuario interactuar con ella para ver los precios de las acciones durante distintos periodos y calcular tanto las ganancias como las pérdidas. Como lo sugieren estos ejemplos, el contenido dinámico se puede generar mediante programas que se ejecuten en el servidor o en el navegador (o en ambos lugares).

En esta sección examinaremos cada uno de estos casos en turno. La situación general es como se muestra en la figura 7-29. Por ejemplo, considere un servicio de mapas que permite al usuario introducir la dirección de una calle y presenta un mapa correspondiente de esa ubicación. Dada una solicitud para una ubicación, el servidor web debe usar un programa para crear una página que muestre el mapa para esa ubicación a partir de una base de datos de calles y demás información geográfica. Esta acción se muestra como los pasos 1 al 3. La solicitud (paso 1) provoca la ejecución de un programa en el servidor. El programa consulta una base de datos para generar la página apropiada (paso 2) y la devuelve al navegador (paso 3).

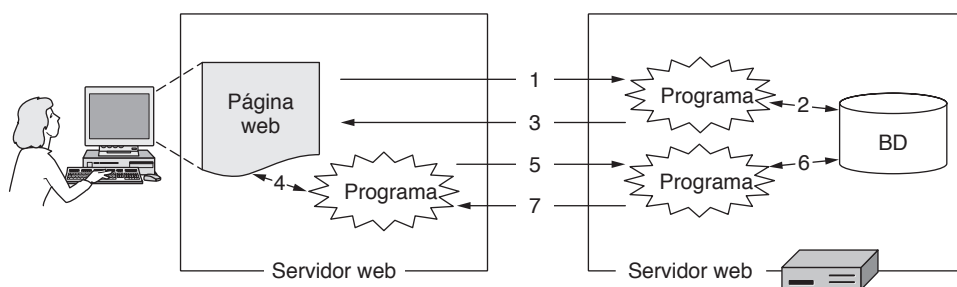


Figura 7-29. Páginas dinámicas.

Sin embargo, hay más sobre el contenido dinámico. La página que se devuelve puede a su vez contener programas que se ejecuten en el navegador. En nuestro ejemplo del mapa, el programa permitiría al usuario buscar rutas y explorar áreas cercanas con distintos niveles de detalle. Actualizaría la página, realizando acercamientos o alejamientos según las indicaciones del usuario (paso 4). Para manejar ciertas interacciones, el programa puede necesitar más datos del servidor. En este caso, enviará una solicitud al servidor (paso 5) para recuperar más información de la base de datos (paso 6) y devolver una respuesta (paso 7). Luego continuará actualizando la página (paso 4). Las solicitudes y respuestas se realizan en segundo plano; el usuario tal vez ni siquiera esté consciente de ellas debido a que por lo general, el URL de la página y el título no cambian. Al incluir programas del lado cliente, la página puede presentar una interfaz más receptiva que cuando se utilizan sólo programas del lado servidor.

Generación de páginas web dinámicas del lado servidor

Ahora veamos el caso de generación de contenido del lado servidor con más detalle. Una situación simple en la que se necesita el procesamiento del lado servidor es cuando se usan formularios. Considere la situación en que el usuario llena el formulario de pedidos WASA de la figura 7-25(b) y hace clic en el botón *Enviar pedido*. Cuando el usuario hace clic, se envía una solicitud al servidor en el URL especificado con el formulario (un *POST* a *http://widget.com/cgi-bin/pedido.cgi* en este caso), junto con el contenido del formulario que llenó el usuario. Hay que proporcionar estos datos a un programa o secuencia de comandos para que los procese. Así, el URL identifica el programa a ejecutar; los datos se proveen al programa en forma de entrada. En este caso, el procesamiento implicaría introducir el pedido en el sistema interno de WASA; actualizar los registros de los clientes y hacer el cargo a la tarjeta de crédito. La página devuelta

por esta solicitud dependerá de lo que ocurra durante el procesamiento. No es fija como una página estática. Si el pedido tiene éxito, la página devuelta podría proporcionar la fecha de entrega esperada. Si no tiene éxito, podría decir que los widgets solicitados están agotados o que la tarjeta de crédito no es válida por algún motivo.

La forma exacta en que el servidor ejecuta un programa en vez de recuperar un archivo depende del diseño del servidor web. Los protocolos web no lo especifican. Esto se debe a que la interfaz puede ser propietaria, por lo que el navegador no necesita conocer los detalles. En lo que a él respecta, simplemente está haciendo una solicitud para obtener una página.

Sin embargo, se han desarrollado varias API estándar para que los servidores web invoquen programas. La existencia de estas interfaces facilita a los desarrolladores el proceso de extender distintos servidores con aplicaciones web. A continuación analizaremos dos API para que el lector se dé una idea de lo que implican.

La primera API es un método para manejar solicitudes de páginas dinámicas, el cual ha estado disponible desde el inicio de la web. Se llama **CGI (Interfaz de Puerta de Enlace Común)**, del inglés *Common Gateway Interface*) y se define en el RFC 3875. CGI provee una interfaz para permitir que los servidores web se comuniquen con programas de soporte y secuencias de comandos que pueden aceptar entradas (por ejemplo, de los formularios) y generar páginas HTML en respuesta. Estos programas pueden estar escritos en cualquier lenguaje que sea conveniente para el desarrollador, por lo general un lenguaje de secuencias de comandos para facilitar el avance. Puede elegir Python, Ruby, Perl o su lenguaje favorito.

Por convención, los programas que se invocan a través de CGI viven en un directorio llamado *cgi-bin*, el cual es visible en el URL. El servidor asocia una solicitud para este directorio con el nombre de un programa, y ejecuta ese programa en un proceso separado. Provee los datos enviados con la solicitud como entrada para el programa. La salida del programa produce una página web que se devuelve al navegador.

En nuestro ejemplo, el programa *pedido.cgi* se invoca con la entrada del formulario codificada, como se muestra en la figura 7-26. Este programa analiza los parámetros y procesa el pedido. Una convención útil es que devuelva el HTML para el formulario de pedido si no se provee un formulario de entrada. De esta manera, se asegurará de conocer la representación del formulario.

La segunda API que analizaremos es algo distinta. La metodología en este caso es incrustar pequeñas secuencias de comandos dentro de las páginas de HTML y hacer que las ejecute el mismo servidor para generar la página. Un lenguaje popular para escribir estas secuencias de comandos es **PHP (Preprocesador de Hipertexto)**, del inglés *Hypertext Preprocessor*). Para usarlo, el servidor tiene que entender PHP, así como un navegador tiene que entender CSS para interpretar las páginas web con hojas de estilo. Por lo general, los servidores identifican las páginas web que contienen PHP con base en la extensión de archivo *php*, en vez de *html* o *htm*.

PHP es más simple de usar que CGI. Como ejemplo de la manera en que trabaja con los formularios, vea la figura 7-30(a). La parte superior de esta figura contiene una página HTML normal con un formulario simple en ella. Esta vez, la etiqueta `<form>` especifica que se debe invocar a *accion.php* para manejar los parámetros cuando el usuario envíe el formulario. La página despliega dos cuadros de texto, uno para solicitar el nombre y el otro para solicitar la edad. Después de que el usuario llena los dos cuadros y envía el formulario, el servidor analiza la cadena de texto que recibe de vuelta (la cual se parece a la de la figura 7-26) y después coloca el nombre en la variable *nombre* junto con la edad en la variable *edad*. Después empieza a procesar el archivo *accion.php*, que se muestra en la figura 7-30(b), como una respuesta. Durante el procesamiento de este archivo se ejecutan los comandos de PHP. Si el usuario escribió “Bárbara” y “24” en los cuadros, el archivo HTML que se envía de vuelta será el que se muestra en la figura 7-30(c). Por lo tanto, manejar formularios es un proceso que se vuelve en extremo simple mediante el uso de PHP.


```

<html>
<body>
<form action="accion.php" method="post">
<p> Por favor introduzca su nombre: <input type="text" name="nombre"> </p>
<p> Por favor introduzca su edad: <input type="text" name="edad"> </p>
<input type="submit">
</form>
</body>
</html>

```

(a)

```

<html>
<body>
<h1> Respuesta: </h1>
Hola <?php echo $nombre; ?>.
Predicción: el siguiente año tendrá <?php echo $edad + 1; ?>
</body>
</html>

```

(b)

```

<html>
<body>
<h1> Respuesta: </h1>
Hola Bárbara.
Predicción: el siguiente año tendrá 33
</body>
</html>

```

(c)

Figura 7-30. (a) Una página web que contiene un formulario. (b) Una secuencia de comandos de PHP para manejar la salida del formulario. (c) Salida de la secuencia de comandos de PHP cuando las entradas son “Bárbara” y “32”, respectivamente.

Aunque PHP es fácil de usar, en realidad es un poderoso lenguaje de programación diseñado para interactuar entre la web y una base de datos del servidor. Tiene variables, cadenas, arreglos y la mayoría de las estructuras de control que se encuentran en C, pero una E/S mucho más poderosa que la instrucción *printf*. PHP es código de fuente abierta, disponible de forma gratuita y muy popular. Se diseñó de manera específica para trabajar bien con Apache, que también es código de fuente abierta y es el servidor web más utilizado en el mundo. Para obtener más información sobre PHP, consulte a Valade (2009).

Ya vimos dos formas diferentes para generar páginas HTML dinámicas: las secuencias de comandos CGI y el PHP incrustado. Existen otras opciones disponibles. **JSP** (*JavaServer Pages*) es similar a PHP, excepto que la parte dinámica se escribe en el lenguaje de programación Java en lugar de PHP. Las páginas que utilizan esta técnica tienen la extensión de archivo *.jsp*. **ASP.NET** (*Active Server Pages .NET*) es la versión de Microsoft de PHP y JavaServer Pages. Utiliza programas escritos en el marco de trabajo (*framework*) de aplicaciones en red .NET propietario de Microsoft para generar el contenido dinámico. Las páginas que utilizan esta técnica tienen la extensión *.aspx*. Por lo general, la selección de una de estas tres técnicas tiene que ver más con políticas (código de fuente abierta contra Microsoft) que con la tecnología, puesto que estos tres lenguajes son más o menos similares.

Generación de páginas web dinámicas en el cliente

Las secuencias de comandos de CGI y PHP resuelven el problema de manejar la entrada y las interacciones con bases de datos en el servidor. Pueden aceptar información entrante de formularios, buscar

información en una o más bases de datos y generar páginas HTML con los resultados. Lo que ninguno de estos lenguajes puede hacer es responder a los movimientos del ratón o interactuar de manera directa con los usuarios. Para esto es necesario tener secuencias de comandos incrustadas en páginas HTML que se ejecuten en la máquina cliente y no en el servidor. Comenzando con HTML 4.0, tales secuencias de comandos son posibles mediante la etiqueta `<script>`. Las tecnologías utilizadas para producir estas páginas web interactivas se conocen en términos generales como **HTML dinámico**.

El lenguaje de secuencias de comandos más popular para el lado cliente es **JavaScript**, por lo que ahora le daremos un vistazo. A pesar de la similitud en los nombres, JavaScript no tiene casi nada que ver con el lenguaje de programación Java. Al igual que otros lenguajes de secuencias de comandos, es un lenguaje de muy alto nivel. Por ejemplo, en una sola línea de JavaScript es posible hacer que aparezca un cuadro de diálogo, esperar a que el usuario introduzca texto mediante el teclado y almacenar la cadena resultante en una variable. Las características de alto nivel como éstas hacen que JavaScript sea ideal para diseñar páginas web interactivas. Por otro lado, el hecho de que tiene más mutaciones que una mosca de fruta atrapada en una máquina de rayos X dificulta de manera considerable el proceso de escribir programas de JavaScript que funcionen en todas las plataformas, pero tal vez algún día se estabilice.

Como ejemplo de un programa de JavaScript, considere el de la figura 7-31. Al igual que el de la figura 7-30, despliega un formulario que pide un nombre y una edad, y después calcula cuántos años tendrá una persona el siguiente año. El cuerpo es casi el mismo que el del ejemplo de PHP; la diferencia principal es la declaración del botón *Submit* y su instrucción de asignación. Ésta indica al navegador que invoque la secuencia de comandos *respuesta* mediante un clic del botón y que la pase al formulario como un parámetro.

Lo que es completamente nuevo aquí es la declaración de la función *respuesta* de JavaScript en el encabezado del archivo HTML, un área que se reserva normalmente para títulos, colores de fondo, etc.

```
<html>
<head>
<script language="javascript" type="text/javascript">
function respuesta(form_prueba) {
    var persona = form_prueba.nombre.value;
    var anios = eval(form_prueba.edad.value) + 1;
    document.open();
    document.writeln("<html> <body>");
    document.writeln("Hola " + persona + "<br>");
    document.writeln("Predicción: el siguiente año tendrá " + anios + " ");
    document.writeln("</body> </html>");
    document.close();
}
</script>
</head>

<body>
<form>
Por favor introduzca su nombre: <input type="text" name="nombre">
<p>
Por favor introduzca su edad: <input type="text" name="edad"> </p>
<p>
<input type="button" value="submit" onclick="respuesta(this.form)">
</form>
</body>
</html>
```

Figura 7-31. Uso de JavaScript para procesar un formulario.

Esta función extrae el valor del campo *nombre* del formulario y lo almacena como una cadena en la variable *persona*. También extrae el valor del campo *edad*, lo convierte en un entero mediante el uso de la función *eval*, le agrega 1 y almacena el resultado en *anios*. A continuación abre un documento para salida, utiliza el método *writeln* para realizar cuatro escrituras en dicho documento y, por último, lo cierra. Este documento es un archivo HTML, como podemos deducir de las diversas etiquetas HTML que contiene. A continuación el navegador despliega el documento en la pantalla.

Es muy importante tener en cuenta que, aunque PHP y JavaScript se ven similares en cuanto a que ambos incrustan código en archivos HTML, se procesan de una manera totalmente distinta. En el ejemplo de PHP de la figura 7-30, después de que el usuario hace clic en el botón *submit*, el navegador recolecta la información en una cadena larga y la envía al servidor como solicitud de una página de PHP. El servidor carga el archivo PHP y ejecuta la secuencia de comandos de PHP que está incrustada para producir una nueva página HTML. Esa página se devuelve al navegador para que la despliegue en pantalla. El navegador ni siquiera puede estar seguro de que fue producida por un programa. Este procesamiento se muestra como los pasos 1 a 4 en la figura 7-32(a).

En el ejemplo de JavaScript de la figura 7-31, al hacer clic en el botón *submit*, el navegador interpreta una función de JavaScript contenida en la página. Todo el trabajo se hace en forma local, dentro del navegador. No hay contacto con el servidor. Este procesamiento se muestra como los pasos 1 y 2 en la figura 7-32(b). Como consecuencia, el resultado se despliega casi al instante, mientras que con PHP puede haber un retraso de varios segundos antes de que llegue el HTML resultante al cliente.

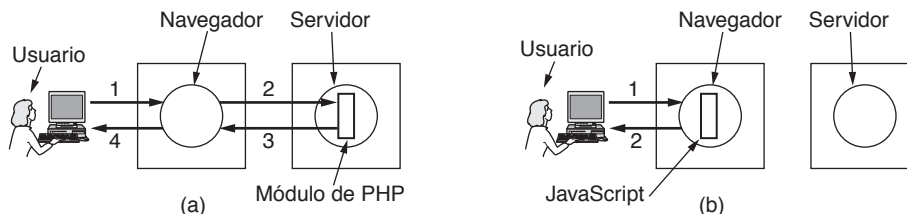


Figura 7-32. (a) Secuencias de comandos del lado servidor con PHP. (b) Secuencias de comandos del lado cliente con JavaScript.

Esta diferencia no significa que JavaScript sea mejor que PHP. Sus usos son completamente distintos. PHP (al igual que JSP y ASP) se utiliza cuando se requiere la interacción con una base de datos en el servidor. JavaScript (junto con otros lenguajes del lado cliente que mencionaremos, como VBScript) se utiliza cuando la interacción es con el usuario en la computadora cliente. Sin duda también es posible combinarlos, como veremos en breve.

JavaScript no es la única forma de crear páginas web muy interactivas. Una alternativa en las plataformas Windows es **VBScript**, que se basa en Visual Basic. Otro método popular entre plataformas es el uso de **applets**. Éstos son pequeños programas de Java compilados en instrucciones de máquina para una computadora virtual, conocida como **JVM (Máquina Virtual de Java, del inglés Java Virtual Machine)**. Los applets se pueden incrustar en páginas HTML (entre `<applet>` y `</applet>`) y pueden ser interpretados por navegadores con soporte para JVM. Como los applets de Java se interpretan en vez de ejecutarse de manera directa, el intérprete de Java puede evitar que hagan “cosas malas”. Por lo menos en teoría. En la práctica, los escritores de applets han descubierto un flujo casi interminable de errores en las bibliotecas de E/S de Java que pueden explotar.

La respuesta de Microsoft a los applets de Java de la empresa Sun fue permitir que las páginas web contuvieran **controles ActiveX**, que son programas compilados para lenguaje de máquina x86 y ejecutados en el hardware. Esta característica los hace muy rápidos y más flexibles que los subprogramas

interpretados de Java, porque pueden hacer cualquier cosa que un programa pueda hacer. Cuando Internet Explorer ve un control ActiveX en una página web, lo descarga, verifica su identidad y lo ejecuta. Sin embargo, la descarga y ejecución de programas extraños aumenta en forma considerable los problemas de seguridad, lo cual veremos en el capítulo 8.

Puesto que casi todos los navegadores pueden interpretar los programas de Java y JavaScript, un diseñador que desee crear una página web muy interactiva puede elegir por lo menos de entre dos técnicas, y si la portabilidad en múltiples plataformas no es importante, también puede elegir ActiveX. Como regla general, los programas de JavaScript son fáciles de escribir, los applets de Java se ejecutan más rápido y los controles ActiveX se ejecutan todavía más rápido. Además, puesto que todos los navegadores implementan exactamente la misma JVM pero no todos implementan la misma versión de JavaScript, los applets de Java son más portables que los programas de JavaScript. Para mayor información sobre JavaScript, hay muchos libros, cada uno con muchas páginas (por lo general, más de 1000). Por ejemplo, puede consultar a Flanagan (2010).

AJAX: JavaScript asíncrono y XML

Las aplicaciones web convincentes necesitan interfaces de usuario receptivas y un acceso transparente a los datos almacenados en servidores web remotos. Las secuencias de comandos en el cliente (por ejemplo, con JavaScript) y en el servidor (por ejemplo, con PHP) son tecnologías básicas que proporcionan algunas piezas de la solución. Esas tecnologías se utilizan frecuentemente con varias otras tecnologías clave en una combinación conocida como **AJAX (Javascript Asíncrono y Xml)**, del inglés *Asynchronous Javascript and Xml*). Muchas aplicaciones web llenas de funcionalidades, como Google Gmail, Maps y Docs, están escritas con AJAX.

El término AJAX es un poco confuso, ya que no es un lenguaje, sino un grupo de tecnologías que trabajan en conjunto para generar aplicaciones web que sean tan receptivas y poderosas como las aplicaciones de escritorio tradicionales. Estas tecnologías son:

1. HTML y CSS para presentar la información como páginas.
2. DOM (Modelo de Objetos de Documento) para modificar partes de las páginas mientras se despliegan en pantalla.
3. XML (Lenguaje de Marcado Extensible) para permitir que los programas intercambien datos de la aplicación con el servidor.
4. Una manera asíncrona para que los programas envíen y recuperen datos XML.
5. JavaScript como lenguaje para enlazar toda esta funcionalidad.

Como ésta es una colección bastante extensa, analizaremos cada una de las piezas para ver lo que contribuye. Ya hemos visto HTML y CSS. Son estándares para describir el contenido y la forma en que se debe desplegar en pantalla. Cualquier programa que pueda producir HTML y CSS, puede usar un navegador web como motor de visualización.

El **DOM (Modelo de Objetos de Documento)**, del inglés *Document Object Model*) es la representación de una página HTML accesible para los programas. Esta representación se estructura como un árbol que refleja la estructura de los elementos de HTML. Por ejemplo, el árbol DOM del HTML de la figura 7-30(a) se proporciona en la figura 7-33. En la raíz hay un elemento *html* que representa a todo el bloque completo de HTML. Este elemento es el padre del elemento *body*, que a su vez es padre de un elemento *form*. El formulario tiene dos atributos que se arrastran al lado derecho, uno para el método del formulario (un *POST*) y otro para la acción del formulario (el URL a solicitar). Este elemento tiene tres hijos, los cuales reflejan las dos etiquetas de párrafo y una etiqueta de entrada que están contenidas dentro del formulario. En la parte inferior del árbol hay hojas que contienen elementos o literales, como las cadenas de texto.

La importancia del modelo DOM es que proporciona a los programas una forma directa de modificar partes de la página. Sin necesidad de volverla a escribir. Sólo hay que reemplazar el nodo que contiene la modificación. Al realizar esta modificación, el navegador actualizará de manera correspondiente la pantalla. Por ejemplo, si en DOM se modifica una imagen en una parte de la página, el navegador actualizará esa imagen sin modificar las demás partes de la página. Ya hemos visto a DOM en acción, con el ejemplo de JavaScript de la figura 7-31 que agrega líneas al elemento *document* para hacer que aparezcan nuevas líneas de texto en la parte inferior de la ventana del navegador. El DOM es un poderoso método para producir páginas que puedan evolucionar.

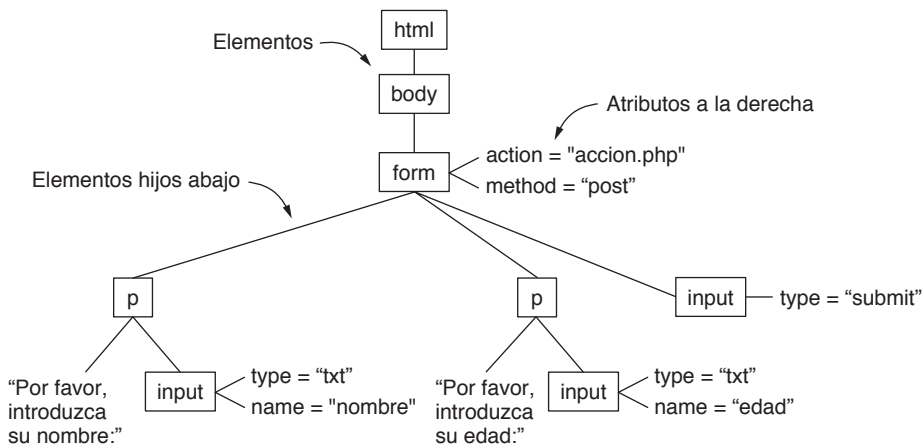


Figura 7-33. El árbol DOM para el HTML de la figura 7-30(a).

La tercera tecnología, **XML (Lenguaje de Marcado Extensible)**, del inglés *eXtensible Markup Language*, es un lenguaje para especificar el contenido estructurado. HTML mezcla el contenido con el formato, puesto que se encarga de la presentación de la información. Sin embargo, a medida que se vuelven más comunes las aplicaciones web aumenta la necesidad de separar el contenido estructurado de su presentación. Por ejemplo, considere un programa que busca en la web el mejor precio de algún libro. Necesita analizar muchas páginas en busca del título y precio del artículo. Si las páginas web están en HTML, es muy difícil que un programa averigüe en dónde están el título y el precio.

Por esta razón, el W3C desarrolló el lenguaje XML (Bray y colaboradores, 2006) para permitir estructurar el contenido web para un procesamiento automatizado. A diferencia del HTML, en XML no hay etiquetas definidas. Cada usuario puede definir sus propias etiquetas. En la figura 7-34 se muestra un ejemplo simple de un documento XML, el cual define una estructura llamada *lista_libros*, que es una lista de libros. Cada uno tiene tres campos: título, autor y año de publicación. Estas estructuras son muy simples. Se permite tener estructuras con campos repetidos (por ejemplo, varios autores), campos opcionales (por ejemplo, el URL del libro de audio) y campos alternativos (por ejemplo, el URL de una librería si se sigue publicando o el URL de un libro de subastas si ya no se publica).

En este ejemplo, cada uno de los tres campos es una entidad indivisible, pero también es posible subdividir los campos todavía más. Por ejemplo, el campo del autor podría haberse organizado de la siguiente manera, para tener un control más detallado en cuanto a la búsqueda y el formato:

```

<autor>
  <primer_nombre> George </primer_nombre>
  <apellido_paterno> Zipf </apellido_paterno>
</autor>

```

Cada campo se puede subdividir en subcampos y subsubcampos, con una profundidad arbitraria.

```
<?xml version="1.0" ?>
<lista_libros>
  <libro>
    <titulo> Human Behavior and the Principle of Least Effort </title>
    <autor> George Zipf </autor>
    <anio> 1949 </anio>
  </libro>
  <libro>
    <titulo> The Mathematical Theory of Communication </title>
    <autor> Claude E. Shannon </autor>
    <anio> 1949 </anio>
  </libro>
  <libro>
    <titulo> Nineteen Eighty-Four </title>
    <autor> George Orwell </autor>
    <anio> 1949 </anio>
  </libro>
</lista_libro>
```

Figura 7-34. Un documento XML simple.

Todo lo que hace el archivo de la figura 7-34 es definir una lista que contiene tres libros. Es adecuada para transportar información entre programas que se ejecutan en navegadores y servidores, pero no dice nada sobre cómo desplegar el documento en pantalla como una página web. Para ello, un programa que consuma la información y juzgue que 1949 fue un excelente año para los libros podría producir un salida HTML donde los títulos se marquen como texto en cursiva. Como alternativa podemos usar un lenguaje llamado **XSLT (Transformaciones del Lenguaje de Hojas de Estilo eXtensible, del inglés *eXtensible Stylesheet Language Transformations*)** para definir cómo se debe transformar el XML en HTML. XSLT es igual que CSS, sólo que mucho más poderoso. A continuación le hablaremos sobre los detalles.

La otra ventaja de expresar los datos en XML en vez de usar HTML es que para los programas es más fácil de analizar. En un principio el HTML se escribía en forma manual (y aún se hace así en muchas ocasiones), por lo que cuando un documento contiene mucho HTML se vuelve algo desordenado. En ocasiones se omiten las etiquetas de cierre, como `</p>`. Otras no tienen una etiqueta de cierre asociada, como `<br>`. Además, hay otras etiquetas que se pueden anidar en forma inapropiada, y el caso de los nombres de etiquetas y atributos puede variar. La mayoría de los navegadores hacen su mejor esfuerzo para tratar de interpretar el documento lo más apegado posible. XML es más estricto y limpio en su definición. Los nombres de las etiquetas y los atributos siempre son en minúscula, las etiquetas siempre se deben cerrar en el orden inverso al orden en que se abrieron (o se debe indicar con claridad cuando se trata de una etiqueta vacía sin su correspondiente etiqueta de cierre), y los valores de los atributos deben ir entre comillas. Esta precisión hace que el análisis sea un proceso sencillo y sin ambigüedades.

Incluso el HTML se está definiendo en términos del XML. A esta metodología se le conoce como **XHTML (Lenguaje de Marcado de Hipertexto Extendido, del inglés *eXtended HyperText Markup Language*)**. En esencia, es una versión “muy quisquillosa” de HTML. Las páginas de XHTML deben formarse de manera estricta a las reglas del XML, o de lo contrario el navegador no las aceptará. No más páginas web de mala calidad ni inconsistencias entre un navegador y otro. Al igual que con el XML,

la intención es producir páginas que los programas (en este caso, las aplicaciones web) puedan procesar mejor. Aunque el XHTML ha estado en funcionamiento desde 1998, ha tardado mucho en popularizarse. Las personas que producen HTML no ven la razón de necesitar XHTML, por lo que se ha retrasado el soporte en los navegadores. Ahora se está definiendo el HTML 5.0, de modo que una página se pueda representar como HTML o XHTML para ayudar en la transición. Eventualmente el XHTML reemplazará al HTML, pero pasará mucho tiempo antes de que se complete esta transición.

XML también ha demostrado ser popular como lenguaje para la comunicación entre los programas. Cuando esta comunicación se lleva a cabo mediante el protocolo HTTP (que describiremos en la siguiente sección), se conoce como servicio web. En particular, **SOAP (Protocolo Simple de Acceso a Objetos)**, del inglés *Simple Object Access Protocol* es una forma de implementar servicios web en la que se realizan llamadas RPC entre los programas de una manera independiente del lenguaje y del sistema. El cliente sólo construye la solicitud como un mensaje XML y lo envía al servidor, mediante el protocolo HTTP. El servidor devuelve una respuesta en forma de un mensaje con formato de XML. De esta manera se pueden comunicar las aplicaciones en plataformas heterogéneas.

Ahora regresemos a AJAX. Nuestro razonamiento es simplemente que XML es un formato útil para intercambiar datos entre programas que se ejecutan en el navegador y el servidor. Sin embargo, para proveer una interfaz receptiva en el navegador mientras se envían o reciben datos, las secuencias de comandos deben ser capaces de realizar operaciones de **E/S asíncrona** para no bloquear la pantalla mientras esperan la respuesta de una solicitud. Por ejemplo, considere un mapa por el que nos podemos desplazar en el navegador. Al recibir la notificación de la acción de desplazamiento, la secuencia de comandos en la página del mapa puede solicitar más datos del mapa al servidor, si la vista del mapa está cerca del borde de los datos. La interfaz no se debe congelar mientras se obtienen esos datos. Dicha interfaz no ganaría premios de parte de los usuarios. En cambio, el desplazamiento debe continuar de manera uniforme. Al llegar los datos, se notifica a la secuencia de comandos para que los pueda utilizar. Si todo sale bien, se obtendrán los nuevos datos del mapa antes de que se requieran. Los navegadores modernos tienen soporte para este modelo de comunicación.

La pieza final del rompecabezas es un lenguaje de secuencias de comandos que mantenga unidos a todos los componentes de AJAX al proveer acceso a la lista anterior de tecnologías. En la mayoría de los casos este lenguaje es JavaScript, pero existen alternativas como VBScript. Anteriormente presentamos un ejemplo simple de JavaScript. Que no lo engañe esta sencillez. JavaScript tiene muchas peculiaridades, pero es un lenguaje de programación completo, con todo el poder de C o Java. Tiene variables, cadenas, arreglos, objetos, funciones y las estructuras de control usuales. También tiene interfaces específicas para el navegador y las páginas web. JavaScript puede rastrear el movimiento del ratón sobre los objetos en la pantalla, gracias a lo cual es fácil hacer que un menú aparezca de repente y conduzca a páginas web vividas. Puede usar DOM para acceder a las páginas, manipular HTML y XML, y llevar a cabo una comunicación asíncrona mediante HTTP.

Antes de dejar el tema de las páginas dinámicas, sintetizaremos brevemente las tecnologías que hemos cubierto hasta ahora, para lo cual las relacionaremos en una sola figura. Se pueden generar páginas web completas al instante mediante diversas secuencias de comandos en la máquina servidor. Las secuencias de comandos se pueden escribir en lenguajes de extensión de servidor, como PHP, JSP o ASP.NET, o se pueden ejecutar como procesos CGI separados y, por ende, es posible escribirlos en cualquier lenguaje. Estas opciones se muestran en la figura 7-35.

Una vez que el navegador recibe estas páginas web, se tratan como páginas normales en HTML, CSS y otros tipos MIME, y simplemente se despliegan en pantalla. Se pueden instalar complementos que se ejecutan en el navegador y aplicaciones ayudantes que se ejecutan fuera del navegador para extender los tipos MIME soportados por el navegador.

También es posible generar contenido dinámico en el lado cliente. Los programas que están incrustados en páginas web se pueden escribir en JavaScript, VBScript, Java y otros lenguajes. Estos

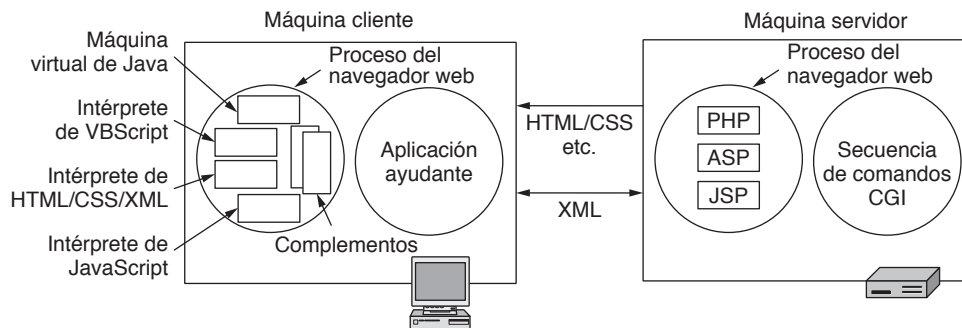


Figura 7-35. Varias tecnologías utilizadas para generar páginas dinámicas.

programas pueden realizar cálculos arbitrarios y actualizar la pantalla. Con AJAX, los programas en las páginas web pueden intercambiar XML y otros tipos de datos en forma asíncrona con el servidor. Este modelo soporta las aplicaciones web ricas en funcionalidad que se ven justo igual que las aplicaciones tradicionales, excepto que se ejecutan dentro del navegador y acceden a la información almacenada en servidores en Internet.

7.3.4 HTTP: el Protocolo de Transferencia de HiperTexto

Ahora que tenemos una idea sobre el contenido web y las aplicaciones, es tiempo de analizar el protocolo que se utiliza para transportar toda esta información entre servidores web y clientes: el **HTTP (Protocolo de Transferencia de HiperTexto)**, del inglés *HyperText Transfer Protocol*), según lo especificado en el RFC 2616.

HTTP es un protocolo simple de solicitud-respuesta que por lo general opera sobre TCP. Especifica qué mensajes pueden enviar los clientes a los servidores, y qué respuestas reciben de estos mensajes. Los encabezados de solicitud y respuesta se proporcionan en ASCII, justo igual que en SMTP. El contenido se proporciona en un formato parecido a MIME, también como el SMTP. Este modelo simple fue en parte responsable del éxito anticipado de la web, ya que simplificó los procesos de desarrollo e implementación.

En esta sección analizaremos las propiedades más importantes del HTTP y su uso en la actualidad. Sin embargo, antes de entrar en detalles cabe mencionar que la forma en que se utiliza en Internet está evolucionando. HTTP es un protocolo de la capa de aplicación, debido a que opera sobre TCP y está muy asociado con la web. Ésta es la razón por la que lo vemos en este capítulo. Sin embargo, en otro sentido el HTTP se está volviendo más parecido a un protocolo de transporte que provee los medios para que los procesos se comuniquen el contenido entre un límite y otro de las distintas redes. Estos procesos no tienen que ser un navegador y un servidor web. Un reproductor de medios podría usar HTTP para comunicarse con un servidor y solicitar información sobre un álbum. El software antivirus podría usarlo para descargar las actualizaciones más recientes. Los desarrolladores podrían usarlo para obtener los archivos de sus proyectos. Los productos de electrónica para el consumidor, como los marcos de fotografías digitales, utilizan con frecuencia un servidor HTTP incrustado como interfaz para el mundo exterior. La comunicación de máquina a máquina se realiza a través de HTTP cada vez con más frecuencia. Por ejemplo, un servidor de una aerolínea podría usar SOAP (una RPC de XML sobre HTTP) para contactarse con un servidor de renta de autos y reservar un vehículo, todo como parte de un paquete vacacional. Es probable que estas tendencias continúen, junto con el uso expandido de HTTP.